INDEX (tap a problem to jump to it)

NEETCODE

01 Arrays
1 Static Arrays
Max Consecutive Ones 3
Remove Element 4
Replace Elements With Greatest Element On Right Side 5
2 Dynamic Arrays
Concatenation of Array 6
Design Dynamic Array (Resizable Array) 8
3 Stacks
Baseball Game 10
Minimum Stack 11
Valid Parentheses 14

02 Linked Lists
1 Singly Linked Lists
Merge Two Sorted Linked Lists 15
Reverse Linked List 17
Design Singly Linked List 19
2 Doubly Linked Lists
Design Linked List 21
Design Browser History 27
3 Queues
Number of Students Unable to Eat Lunch 30
Implement Stack Using Queues 31
Design Double-ended Queue 33

03 Recursion
2 Fibonacci Sequence
Climbing Stairs 35

04 Sorting
1 Insertion Sort
Insertion Sort 37
2 Merge Sort
Merge Sort 38
Merge Sorted Array 40
Merge K Sorted Linked Lists 41
3 Quick Sort
Quick Sort 42
4 Bucket Sort
Sort Colors 44

05 Binary Search
1 Search Array
Search a 2D Matrix 45
2 Search Range
Guess Number Higher or Lower 46
Koko Eating Bananas 47

06 Trees
2 BST Insert and Remove
Insert into a Binary Search Tree 48
Delete Node in a BST 49
3 Depth-First Search
Construct Binary Tree from Preorder and Inorder Traversal 51
Kth Smallest Integer in BST 53
4 Breadth-First Search
Binary Tree Level Order Traversal 55
Binary Tree Right Side View 57

07 Backtracking
Cheatsheet 58
1 Tree Maze
Path Sum 61
Subsets 62
Combination Target Sum 65

08 Heap - Priority Queue
2 Heapify
K Closest Points to Origin 68

09 Hashing
1 Hash Usage
Two Integer Sum 69
LRU Cache 70

10 Graphs
Cheatsheet 73
1 Matrix DFS
Matrix Depth-First Search: Count Paths 75
Count Number of Islands 76
Max Area of Island 79
Flood Fill 81
2 Matrix BFS
Matrix Breadth-First Search: Shortest Path 82
Shortest Path in Binary Matrix 84
Rotting Fruit 86

13 Sliding Window
Best Time to Buy and Sell Stock 88
Longest Substring Without Repeating Characters 89

CODILITY

01 Arrays
Cyclic Rotation 90
Max Counters 91

02 Linked Lists
Arr List Len 93

09 Hashing
Perm Missing Elem 96
Frog River One 97
Missing Integer 98

12 Bit Manipulation
Binary Gap 99

13 Sliding Window
Count Bounded Slices 100
Count Distinct Slices 101

14 Prefix Sums
Tape Equilibrium 103
Genomic Range Query 104
Passing Cars 106
Min Avg Two Slice 108

```python
### max-consecutive-ones.py

"""
Max Consecutive Ones (NeetCode)
===============================

Given a binary array nums, return the maximum number of consecutive 1's in the
array.

Example 1:
    Input:  nums = [1,1,0,1,1,1]
    Output: 3

Example 2:
    Input:  nums = [1,0,1,1,0,1]
    Output: 2

Constraints:
    1 <= nums.length <= 100,000
    nums[i] is either 0 or 1.
"""

class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        max_count = 0
        count = 0

        for num in nums:
            if num==1:
                count+=1
                max_count = max(max_count, count)
            else:
                count = 0

        return max_count
```

```python
### remove-element.py

"""
Remove Element (NeetCode)
=========================

You are given an integer array nums and an integer val. Your task is to remove
all occurrences of val from nums in-place.

After removing all occurrences of val, return the number of remaining elements,
say k, such that the first k elements of nums do not contain val.

Note:
    - The order of the elements which are not equal to val does not matter.
    - It is not necessary to consider elements beyond the first k positions.
    - To be accepted, the first k elements of nums must contain only elements
      not equal to val.

Return k as the final result.

Example 1:
    Input:  nums = [1,1,2,3,4], val = 1
    Output: [2,3,4]
    Explanation: You should return k = 3 as we have 3 elements which are not
    equal to val = 1.

Example 2:
    Input:  nums = [0,1,2,2,3,0,4,2], val = 2
    Output: [0,1,3,0,4]
    Explanation: You should return k = 5 as we have 5 elements which are not
    equal to val = 2.

Constraints:
    0 <= nums.length <= 100
    0 <= nums[i] <= 50
    0 <= val <= 100
"""

class Solution:
    def removeElement(self, nums: List[int], val: int) -> int:
        k = 0

        # using two pointers
        # i iterates through the array
        # k keeps track of where to place the "non-value"
        for i in range(len(nums)):
            if nums[i]!=val:
                nums[k] = nums[i]
                k+=1
        return k
```

```python
### replace-elements-with-greatest-element-on-right-side.py

"""
Replace Elements With Greatest Element On Right Side (NeetCode)
===============================================================

You are given an array arr, replace every element in that array with the
greatest element among the elements to its right, and replace the last element
with -1.

After doing so, return the array.

Example 1:
    Input:  arr = [2,4,5,3,1,2]
    Output: [5,5,3,2,2,-1]

Example 2:
    Input:  arr = [3,3]
    Output: [3,-1]

Constraints:
    1 <= arr.length <= 10,000
    1 <= arr[i] <= 100,000
"""

class Solution:
    def replaceElements(self, arr: List[int]) -> List[int]:
        # traverse right-to-left,
        # tracking the max of everything seen so far

        max_right = -1

        for i in range(len(arr)-1, -1, -1):
            # stage the next max BEFORE overwriting arr[i],
            # otherwise we lose arr[i]'s original value
            new_max = max(max_right, arr[i])
            arr[i] = max_right
            max_right = new_max

        return arr
```

```
### concatenation-of-array-append.py

"""
Concatenation of Array (NeetCode)
=================================

You are given an integer array nums of length n. Create an array ans of length
2n where ans[i] == nums[i] and ans[i + n] == nums[i] for 0 <= i < n
(0-indexed).

Specifically, ans is the concatenation of two nums arrays.

Return the array ans.

Example 1:
    Input:  nums = [1,4,1,2]
    Output: [1,4,1,2,1,4,1,2]

Example 2:
    Input:  nums = [22,21,20,1]
    Output: [22,21,20,1,22,21,20,1]

Constraints:
    1 <= nums.length <= 1000
    1 <= nums[i] <= 1000
"""

class Solution:
    def getConcatenation(self, nums: List[int]) -> List[int]:
        # nums_len = len(nums)
        # new_nums = [0]*2*nums_len

        # for i in range(nums_len):
        #     # first copy
        #     new_nums[i] = nums[i]
        #     # second copy, offset by nums_len
        #     new_nums[i+nums_len] = nums[i]

        # return new_nums


        # NEETCODE easier soln
        ans = []
        for _ in range(2):
            for i in range(len(nums)):
                ans.append(nums[i])

        return ans
```

```python
### concatenation-of-array-preallocated.py

"""
Concatenation of Array (NeetCode) -- another approach
Problem statement: see concatenation-of-array-append.py
"""

class Solution:
    def getConcatenation(self, nums: List[int]) -> List[int]:
        nums_len = len(nums)
        new_nums = [0]*2*nums_len

        for i in range(nums_len):
            new_nums[i] = nums[i]
            new_nums[i+nums_len] = nums[i]

        return new_nums
```

```
### dynamic-array.py

"""
Design Dynamic Array (Resizable Array) (NeetCode)
=================================================

Design a Dynamic Array (aka a resizable array) class, such as an ArrayList in
Java or a vector in C++.

Your DynamicArray class should support the following operations:
    - DynamicArray(int capacity) will initialize an empty array with a capacity
      of capacity, where capacity > 0.
    - int get(int i) will return the element at index i. Assume that index i
      is valid.
    - void set(int i, int n) will set the element at index i to n. Assume that
      index i is valid.
    - void pushback(int n) will push the element n to the end of the array.
    - int popback() will pop and return the element at the end of the array.
      Assume that the array is non-empty.
    - void resize() will double the capacity of the array.
    - int getSize() will return the number of elements in the array.
    - int getCapacity() will return the capacity of the array.

If we call void pushback(int n) but the array is full, we should resize the
array first.

Example 1:
    Input:  ["Array", 1, "getSize", "getCapacity"]
    Output: [null, 0, 1]

Example 2:
    Input:  ["Array", 1, "pushback", 1, "getCapacity", "pushback", 2,
             "getCapacity"]
    Output: [null, null, 1, null, 2]

Example 3:
    Input:  ["Array", 1, "getSize", "getCapacity", "pushback", 1, "getSize",
             "getCapacity", "pushback", 2, "getSize", "getCapacity", "get", 1,
             "set", 1, 3, "get", 1, "popback", "getSize", "getCapacity"]
    Output: [null, 0, 1, null, 1, 1, null, 2, 2, 2, null, 3, 3, 1, 2]

Note:
    The index i provided to get(int i) and set(int i) is guaranteed to be
    greater than or equal to 0 and less than the number of elements in the
    array.
"""

class DynamicArray:

    def __init__(self, capacity: int):
        # Max capacity of the backing array
        self.capacity = capacity
        # Number of elements currently stored (<= capacity)
        self.length = 0
        # Backing storage
        self.arr = [0] * capacity

    # Get value at i-th index
    def get(self, i: int) -> int:
        return self.arr[i]

    # Set value n at i-th index
    def set(self, i: int, n: int) -> None:
        self.arr[i] = n

    # Push n to the end of the array
    def pushback(self, n: int) -> None:
        # Double capacity if we're full
        if self.length == self.capacity:
            self.resize()
        # Place n at next empty slot and grow logical size
        self.arr[self.length] = n
        self.length += 1

    # Pop and return the last element
    def popback(self) -> int:
        # Shrink logical size, then return the removed element
```

```
        self.length -= 1
        return self.arr[self.length]

    # Double the capacity of the array
    def resize(self) -> None:
        self.capacity = 2 * self.capacity
        new_arr = [0] * self.capacity
        # Copy only the used elements into the new array
        for i in range(self.length):
            new_arr[i] = self.arr[i]
        self.arr = new_arr

    # Return the number of elements in the array
    def getSize(self) -> int:
        return self.length

    # Return the current capacity of the array
    def getCapacity(self) -> int:
        return self.capacity
```

```
### baseball-game.py

"""
Baseball Game (NeetCode)
========================

You are keeping the scores for a baseball game with strange rules. At the
beginning of the game, you start with an empty record.

Given a list of strings operations, where operations[i] is the ith operation
you must apply to the record and is one of the following:
    - An integer x: Record a new score of x.
    - '+': Record a new score that is the sum of the previous two scores.
    - 'D': Record a new score that is the double of the previous score.
    - 'C': Invalidate the previous score, removing it from the record.

Return the sum of all the scores on the record after applying all the
operations.

Note: The test cases are generated such that the answer and all intermediate
calculations fit in a 32-bit integer and that all operations are valid.

Example 1:
    Input:  ops = ["1","2","+","C","5","D"]
    Output: 18
    Explanation:
    "1" - Add 1 to the record, record = [1].
    "2" - Add 2 to the record, record = [1, 2].
    "+" - Add 1 + 2 = 3 to the record, record = [1, 2, 3].
    "C" - Invalidate and remove the previous score, record = [1, 2].
    "5" - Add 5 to the record, record = [1, 2, 5].
    "D" - Add 2 * 5 = 10 to the record, record = [1, 2, 5, 10].
    The total sum is 1 + 2 + 5 + 10 = 18.

Example 2:
    Input:  ops = ["5","D","+","C"]
    Output: 15
    Explanation:
    "5" - Add 5 to the record, record = [5].
    "D" - Add 2 * 5 = 10 to the record, record = [5, 10].
    "+" - Add 5 + 10 = 15 to the record, record = [5, 10, 15].
    "C" - Invalidate and remove the previous score, record = [5, 10].
    The total sum is 5 + 10 = 15.

Constraints:
    1 <= operations.length <= 1000
    operations[i] is "C", "D", "+", or a string representing an integer in the
    range [-30,000, 30,000].
    For operation "+", there will always be at least two previous scores on
    the record.
    For operations "C" and "D", there will always be at least one previous
    score on the record.
"""

class Solution:
    def calPoints(self, operations: List[str]) -> int:
        stack = []

        for op in operations:
            if op == "C":
                stack.pop()
            elif op == "D":
                last_el = stack[-1]
                doubled = 2*last_el
                stack.append(doubled)
            elif op == "+":
                new_el = stack[-1]+stack[-2]
                stack.append(new_el)
            else:
                stack.append(int(op))

        return sum(stack)
```

```
### minimum-stack-v1.py

"""
Minimum Stack (NeetCode)
========================

Design a stack class that supports the push, pop, top, and getMin operations.
    - MinStack() initializes the stack object.
    - void push(int val) pushes the element val onto the stack.
    - void pop() removes the element on the top of the stack.
    - int top() gets the top element of the stack.
    - int getMin() retrieves the minimum element in the stack.

Each function should run in O(1) time.

Example 1:
    Input:  ["MinStack", "push", 1, "push", 2, "push", 0, "getMin", "pop",
             "top", "getMin"]
    Output: [null, null, null, null, 0, null, 2, 1]

    Explanation:
    MinStack minStack = new MinStack();
    minStack.push(1);
    minStack.push(2);
    minStack.push(0);
    minStack.getMin(); // return 0
    minStack.pop();
    minStack.top();    // return 2
    minStack.getMin(); // return 1

Constraints:
    -2^31 <= val <= 2^31 - 1.
    pop, top and getMin will always be called on non-empty stacks.
"""

class MinStack:

    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        if len(self.min_stack)>0:
            self.min_stack.append(min(val, self.min_stack[-1]))
        else:
            self.min_stack.append(val)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

```python
### minimum-stack-v2.py

"""
Minimum Stack (NeetCode) -- another approach
Problem statement: see minimum-stack-v1.py
"""

class MinStack:

    def __init__(self):
        # whole logic is to have two stacks for O(1) time complexity.
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        if len(self.min_stack)>0:
            self.min_stack.append(min(val, self.min_stack[-1]))
        else:
            self.min_stack.append(val)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

```python
### minimum-stack-v3.py

"""
Minimum Stack (NeetCode) -- another approach
Problem statement: see minimum-stack-v1.py
"""

class MinStack:

    def __init__(self):
        # whole logic is to have two stacks for O(1) time complexity.
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        if len(self.min_stack)>0:
            # min_stack[i] holds the min of stack[0..i], so the top of
            # min_stack is always the current global minimum.
            self.min_stack.append(min(val, self.min_stack[-1]))
        else:
            self.min_stack.append(val)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

```python
### validate-parentheses.py

"""
Valid Parentheses (NeetCode)
============================

You are given a string s consisting of the following characters: '(', ')',
'{', '}', '[' and ']'.

The input string s is valid if and only if:
    1. Every open bracket is closed by the same type of close bracket.
    2. Open brackets are closed in the correct order.
    3. Every close bracket has a corresponding open bracket of the same type.

Return true if s is a valid string, and false otherwise.

Example 1:
    Input:  s = "[]"
    Output: true

Example 2:
    Input:  s = "([{}])"
    Output: true

Example 3:
    Input:  s = "[(])"
    Output: false
    Explanation: The brackets are not closed in the correct order.

Constraints:
    1 <= s.length <= 1000
"""

class Solution:
    def isValid(self, s: str) -> bool:
        stack = []

        # for strings like "["
        if len(s)%2!=0:
            return False

        for char in s:
            if char in "([{":
                stack.append(char)
            elif char in "}])":
                if len(stack) > 0:
                    # for strings like "]]"
                    popped = stack.pop()
                    popped_char_string_pair = popped+char
                    if popped_char_string_pair not in ["()", "{}", "[]"] :
                        return False
                else:
                    return False
            else:
                None

        return len(stack)==0
```

```
### merge-two-sorted-linked-lists-iterative.py

"""
Merge Two Sorted Linked Lists (NeetCode)
========================================

You are given the heads of two sorted linked lists list1 and list2.

Merge the two lists into one sorted linked list and return the head of the new
sorted linked list.

The new list should be made up of nodes from list1 and list2.

Example 1:
    Input:  list1 = [1,2,4], list2 = [1,3,5]
    Output: [1,1,2,3,4,5]

Example 2:
    Input:  list1 = [], list2 = [1,2]
    Output: [1,2]

Example 3:
    Input:  list1 = [], list2 = []
    Output: []

Constraints:
    0 <= The length of the each list <= 100.
    -100 <= Node.val <= 100
"""

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListN
ode]:
        # ITERATIVE
        # Time complexity O(n+m)
        # Space complexity O(1)

        dummy = node = ListNode()

        while list1 and list2:
            if list1.val < list2.val:
                node.next = ListNode(list1.val)
                list1 = list1.next
            else:
                node.next = ListNode(list2.val)
                list2 = list2.next
            node = node.next

        node.next = list1 or list2

        return dummy.next
```

```python
### merge-two-sorted-linked-lists-recursive.py

"""
Merge Two Sorted Linked Lists (NeetCode) -- another approach
Problem statement: see merge-two-sorted-linked-lists-iterative.py
"""

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) -> Optional[ListN
ode]:
        # RECURSIVE
        # Time complexity O(n+m)
        # Space complexity O(n+m)

        # BASE CASE 1: list1 is exhausted -> the rest of the merged list is just list2
        if list1 is None:
            return list2

        # BASE CASE 2: list2 is exhausted -> the rest of the merged list is just list1
        if list2 is None:
            return list1

        if list1.val < list2.val:
            # list1 wins this spot. Recurse with list1's remainder vs the whole list2.
            # Whatever comes back is appended right after list1.
            list1.next = self.mergeTwoLists(list1.next, list2)
            return list1
        else:
            # list2 wins this spot (ties go to list2, which is fine).
            list2.next = self.mergeTwoLists(list2.next, list1)
            return list2

# Let's trace list1 = [1, 2, 4] and list2 = [1, 3, 5].

# Call 1: merge([1,2,4], [1,3,5])
#   1 < 1? No  -> else branch: list2 (val=1) wins
#   list2.next = merge([1,2,4], [3,5])   <- note list2 advanced
#   return list2 (val=1)

#   Call 2: merge([1,2,4], [3,5])
#     1 < 3? Yes -> list1 (val=1) wins
#     list1.next = merge([2,4], [3,5])
#     return list1 (val=1)

#     Call 3: merge([2,4], [3,5])
#       2 < 3? Yes -> list1 (val=2) wins
#       list1.next = merge([4], [3,5])
#       return list1 (val=2)

#       Call 4: merge([4], [3,5])
#         4 < 3? No -> else: list2 (val=3) wins
#         list2.next = merge([4], [5])
#         return list2 (val=3)

#         Call 5: merge([4], [5])
#           4 < 5? Yes -> list1 (val=4) wins
#           list1.next = merge([], [5])
#           return list1 (val=4)

#           Call 6: merge([], [5])
#             BASE CASE: list1 is None -> return list2 ([5])

#     Unwinding the stack (each call attaches its result to .next):

#     Call 5 returns: 4 -> 5
#     Call 4 returns: 3 -> 4 -> 5
#     Call 3 returns: 2 -> 3 -> 4 -> 5
#     Call 2 returns: 1 -> 2 -> 3 -> 4 -> 5
#     Call 1 returns: 1 -> 1 -> 2 -> 3 -> 4 -> 5
```

```python
### reverse-a-linked-list-iterative.py

"""
Reverse Linked List (NeetCode)
==============================

Given the beginning of a singly linked list head, reverse the list, and return
the new beginning of the list.

Example 1:
    Input:  head = [0,1,2,3]
    Output: [3,2,1,0]

Example 2:
    Input:  head = []
    Output: []

Constraints:
    0 <= The length of the list <= 1000.
    -1000 <= Node.val <= 1000
"""

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # ITERATIVE using TWO pointers
        prev = None
        curr = head

        while curr:
            # stage the next node BEFORE we overwrite curr.next,
            # otherwise we lose the rest of the list.
            temp = curr.next
            curr.next = prev
            prev = curr
            curr = temp

        # when curr is None, the loop is done.
        # prev now sits on the original LAST node = new front.
        return prev
```

```python
### reverse-a-linked-list-recursive.py

"""
Reverse Linked List (NeetCode) -- another approach
Problem statement: see reverse-a-linked-list-iterative.py
"""

# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next

class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        # RECURSION
        new_head = head

        if not head or not head.next:
            return head

        # ------------------------------------------------------------
        # RECURSIVE STEP:
        #   Trust the recursion: "reverse everything AFTER me."
        #   new_head will be the last node (3), which is the new front.
        # ------------------------------------------------------------
        new_head = self.reverseList(head.next)

        # The two lines that do everything
        head.next.next = head # flip the arrow
        head.next = None # cut the old forward link to avoid a cycle

        return new_head

# EXPLANATION
# reverseList(1)
#    calls reverseList(2)
#       calls reverseList(3)
#          base case: 3.next is None -> return 3   <- new_head = 3


# Back in reverseList(2):   new_head = 3
#    2.next.next = 2   ->   3.next = 2     (3 now points back to 2)
#    2.next = None     ->   breaks 2 -> 3
#    return 3
#    List so far: 3 -> 2 -> None

# Back in reverseList(1):   new_head = 3
#    1.next.next = 1   ->   2.next = 1     (2 now points back to 1)
#    1.next = None     ->   breaks 1 -> 2
#    return 3
#    Final list: 3 -> 2 -> 1 -> None
```

```
### singly-linked-list.py

"""
Design Singly Linked List (NeetCode)
====================================

Design a Singly Linked List class.

Your LinkedList class should support the following operations:
    - LinkedList() will initialize an empty linked list.
    - int get(int i) will return the value of the ith node (0-indexed). If the
      index is out of bounds, return -1.
    - void insertHead(int val) will insert a node with val at the head of the
      list.
    - void insertTail(int val) will insert a node with val at the tail of the
      list.
    - bool remove(int i) will remove the ith node (0-indexed). If the index is
      out of bounds, return false, otherwise return true.
    - int[] getValues() return an array of all the values in the linked list,
      ordered from head to tail.

Example 1:
    Input:  ["insertHead", 1, "insertTail", 2, "insertHead", 0, "remove", 1,
             "getValues"]
    Output: [null, null, null, true, [0, 2]]

Example 2:
    Input:  ["insertHead", 1, "insertHead", 2, "get", 5]
    Output: [null, null, -1]

Note:
    The index int i provided to get(int i) and remove(int i) is guaranteed to
    be greater than or equal to 0.
"""

class Node:
    def __init__(self,val):
        self.val = val
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = Node(0)
        self.size = 0

    def get(self, index: int) -> int:
        if self.size <= index:
            return -1

        curr = self.head.next
        for i in range(index):
            curr = curr.next
        return curr.val

    def insertHead(self, val: int) -> None:
        curr = self.head
        new_node = Node(val)
        new_node.next = curr.next
        curr.next = new_node
        self.size += 1

    def insertTail(self, val: int) -> None:
        curr = self.head
        for i in range(self.size):
            curr = curr.next

        new_node = Node(val)
        curr.next = new_node
        self.size += 1

    def remove(self, index: int) -> bool:
        if self.size <= index:
            return False

        curr = self.head
        for i in range(index):
            curr = curr.next
```

```
        curr.next = curr.next.next
        self.size -= 1

        return True

    def getValues(self) -> List[int]:
        ans = []
        curr = self.head.next
        for i in range(self.size):
            ans.append(curr.val)
            curr = curr.next

        return ans
```

```
### design-linked-list-doubly.py

"""
Design Linked List (NeetCode)
=============================

Design your implementation of the linked list. You can choose to use a singly
or doubly linked list.

A node in a singly linked list should have two attributes: val and next. val is
the value of the current node, and next is a pointer/reference to the next
node. If you want to use the doubly linked list, you will need one more
attribute prev to indicate the previous node in the linked list. Assume all
nodes in the linked list are 0-indexed.

Implement the MyLinkedList class:
    - MyLinkedList() Initializes the MyLinkedList object.
    - int get(int index) Get the value of the indexth node in the linked list.
      If the index is invalid, return -1.
    - void addAtHead(int val) Add a node of value val before the first element
      of the linked list. After the insertion, the new node will be the first
      node of the linked list.
    - void addAtTail(int val) Append a node of value val as the last element of
      the linked list.
    - void addAtIndex(int index, int val) Add a node of value val before the
      indexth node in the linked list. If index equals the length of the
      linked list, the node will be appended to the end of the linked list. If
      index is greater than the length, the node will not be inserted.
    - void deleteAtIndex(int index) Delete the indexth node in the linked list,
      if the index is valid.

Example 1:
    Input:  ["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get",
             "deleteAtIndex", "get"]
            [[], [1], [3], [1, 2], [1], [1], [1]]
    Output: [null, null, null, null, 2, null, 3]

    Explanation:
    MyLinkedList myLinkedList = new MyLinkedList();
    myLinkedList.addAtHead(1);
    myLinkedList.addAtTail(3);
    myLinkedList.addAtIndex(1, 2);  // linked list becomes 1->2->3
    myLinkedList.get(1);            // return 2
    myLinkedList.deleteAtIndex(1);  // now the linked list is 1->3
    myLinkedList.get(1);            // return 3

Constraints:
    0 <= index, val <= 1000
    Please do not use the built-in LinkedList library.
    At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex
    and deleteAtIndex.
"""

class Node:
    def __init__(self, val):
        self.val = val
        self.next = None
        self.prev = None

class MyLinkedList:
    # doubly LL
    def __init__(self):
        self.left = Node(0)             # left sentinel (dummy head)
        self.right = Node(0)            # right sentinel (dummy tail)
        self.left.next = self.right
        self.right.prev = self.left

    def get(self, index: int) -> int:
        curr = self.left.next           # first real node

        while curr and index>0:
            index -= 1
            curr = curr.next

        # must exclude right sentinel (its val is a placeholder)
        if curr and curr!= self.right:
            return curr.val
```

```
        return -1

    def addAtHead(self, val: int) -> None:
        new_node, next, prev = Node(val), self.left.next, self.left
        new_node.next = next
        new_node.prev = prev
        next.prev = new_node
        prev.next = new_node

    def addAtTail(self, val: int) -> None:
        # right sentinel gives O(1) access to the tail
        new_node, next, prev = Node(val), self.right, self.right.prev
        new_node.next = next
        new_node.prev = prev
        next.prev = new_node
        prev.next = new_node

    def addAtIndex(self, index: int, val: int) -> None:
        curr = self.left.next
        while curr and index>0:
            index -= 1
            curr = curr.next

        # index==0 means we landed on target; curr.prev gives predecessor in O(1)
        if curr and index==0:
            new_node, next, prev = Node(val), curr, curr.prev
            new_node.next = next
            new_node.prev = prev
            next.prev = new_node
            prev.next = new_node

    def deleteAtIndex(self, index: int) -> None:
        curr = self.left.next
        while curr and index>0:
            index -= 1
            curr = curr.next

        # extra curr != self.right: can't delete a sentinel
        if curr and curr != self.right and index==0:
            next, prev = curr.next, curr.prev
            next.prev = prev
            prev.next = next
```

```python
### design-linked-list-singly-while.py

"""
Design Linked List (NeetCode) -- another approach
Problem statement: see design-linked-list-doubly.py
"""

class ListNode:
    def __init__(self, val):
        self.val = val
        self.next = None

class MyLinkedList:
    # SINGLE LINKED LIST USING WHILE
    # self.head is a dummy (sentinel) node; real data starts at self.head.next.
    # The dummy means insert/delete at index 0 needs no special case:
    # there is always a "previous" node.
    # ** curr = self.head.next only in case of get method.
    def __init__(self):
        self.head = ListNode(0)
        self.size = 0

    def is_empty(self): # helper func
        return (self.size == 0)

    def print_ll(self): # helper func
        if not self.is_empty():
            curr = self.head.next               # skip dummy
            while curr:
                print(curr.val, end="->")
                curr = curr.next
            print("None")
        else:
            print("empty list")

    def get(self, index: int) -> int:
        # valid indices are 0..size-1
        if index >= self.size:
            return -1

        # start at the first real node; after `index` steps curr IS the target
        curr = self.head.next
        # `while curr` (not curr.next) so the loop can land on the last node
        while curr:
            if index == 0:                      # check BEFORE decrementing
                return curr.val
            index -= 1
            curr = curr.next

    def addAtHead(self, val: int) -> None:
        # insert right after the dummy
        new_node = ListNode(val)
        new_node.next = self.head.next          # link new node to old first node
        self.head.next = new_node               # dummy now points to new node
        self.size += 1

    def addAtTail(self, val: int) -> None:
        new_node = ListNode(val)
        curr = self.head
        while curr.next:
            curr = curr.next
        # above loop breaks at last node as curr.next is None
        curr.next = new_node
        self.size += 1

    def addAtIndex(self, index: int, val: int) -> None:
        # valid indices are 0..size (index == size means append at tail)
        if index > self.size:
            return

        new_node = ListNode(val)
        # start at the dummy; after `index` steps curr is the node BEFORE the insert position
        curr = self.head

        # `while curr` (not curr.next): when index == size, the node before
        # the insert position is the tail, whose .next is None.
        # `while curr.next` would exit before checking index == 0.
```

```
        while curr:
            if index == 0:                      # check BEFORE decrementing
                new_node.next = curr.next       # new node points to old node at this index
                curr.next = new_node            # previous node points to new node
                self.size += 1
                return
            index -= 1
            curr = curr.next

    def deleteAtIndex(self, index: int) -> None:
        # valid indices are 0..size-1 (index == size falls through the loop harmlessly)
        if index > self.size:
            return

        # start at the dummy; after `index` steps curr is the node BEFORE the target
        curr = self.head
        # `while curr.next`: there must be a node after curr to delete.
        # Stopping at the tail is correct because there is nothing after it.
        while curr.next:
            if index == 0:                      # check BEFORE decrementing
                curr.next = curr.next.next      # bypass target; it gets garbage collected
                self.size -= 1
                return
            index -= 1
            curr = curr.next
```

```python
### design-linked-list-singly-for.py

"""
Design Linked List (NeetCode) -- another approach
Problem statement: see design-linked-list-doubly.py
"""

class Node:
    def __init__(self,val):
        self.val = val
        self.next = None

class MyLinkedList:
    # SIMPLE LINKED LIST WITH FOR
    def __init__(self):
        self.head = Node(0)
        self.size = 0

    def is_empty(self):
        return (self.size==0)

    def print_ll(self):
        curr = self.head.next
        for i in range(self.size):
            print(curr.val, end = "=>")
            curr = curr.next
        print()

    def get(self, index: int) -> int:
        if index >= self.size:
            return -1

        curr = self.head.next
        for i in range(index):
            curr = curr.next

        return curr.val

    def addAtHead(self, val: int) -> None:
        new_node = Node(val)
        new_node.next = self.head.next
        self.head.next = new_node
        self.size += 1

    def addAtTail(self, val: int) -> None:
        new_node = Node(val)

        curr = self.head
        for i in range(self.size):
            curr = curr.next
        curr.next = new_node
        self.size += 1

    def addAtIndex(self, index: int, val: int) -> None:
        if index > self.size:
            return

        new_node = Node(val)
        curr = self.head

        for i in range(index):
            curr = curr.next

        new_node.next = curr.next
        curr.next = new_node
        self.size += 1

    def deleteAtIndex(self, index: int) -> None:
        if index >= self.size:
            return

        curr = self.head
        for i in range(index):
            curr = curr.next

        curr.next = curr.next.next
        self.size -= 1
```

```
# Your MyLinkedList object will be instantiated and called as such:
# obj = MyLinkedList()
# param_1 = obj.get(index)
# obj.addAtHead(val)
# obj.addAtTail(val)
# obj.addAtIndex(index,val)
# obj.deleteAtIndex(index)
```

```
### design-browser-history-array.py

"""
Design Browser History (NeetCode)
==================================

You have a browser of one tab where you start on the homepage and you can visit
another url, get back in the history number of steps or move forward in the
history number of steps.

Implement the BrowserHistory class:
    - BrowserHistory(string homepage) Initializes the object with the homepage
      of the browser.
    - void visit(string url) Visits url from the current page. It clears up all
      the forward history.
    - string back(int steps) Move steps back in history. If you can only return
      x steps in the history and steps > x, you will return only x steps.
      Return the current url after moving back in history at most steps.
    - string forward(int steps) Move steps forward in history. If you can only
      forward x steps in the history and steps > x, you will forward only x
      steps. Return the current url after forwarding in history at most steps.

Example 1:
    Input:
    ["BrowserHistory", "visit", "visit", "visit", "back", "back", "forward",
     "visit", "forward", "back", "back"]
    [["neetcode.io"], ["google.com"], ["facebook.com"], ["youtube.com"], [1],
     [1], [1], ["linkedin.com"], [2], [2], [7]]

    Output:
    [null, null, null, null, "facebook.com", "google.com", "facebook.com",
     null, "linkedin.com", "google.com", "neetcode.io"]

    Explanation:
    BrowserHistory browserHistory = new BrowserHistory("neetcode.io");
    browserHistory.visit("google.com");    // neetcode.io -> google.com
    browserHistory.visit("facebook.com"); // google.com -> facebook.com
    browserHistory.visit("youtube.com");  // facebook.com -> youtube.com
    browserHistory.back(1);     // back to facebook.com, return "facebook.com"
    browserHistory.back(1);     // back to google.com, return "google.com"
    browserHistory.forward(1); // forward to facebook.com, return "facebook.com"
    browserHistory.visit("linkedin.com"); // facebook.com -> linkedin.com
    browserHistory.forward(2); // can't move forward, return "linkedin.com"
    browserHistory.back(2);     // back two steps to google.com, return it
    browserHistory.back(7);     // can only go back one step, return "neetcode.io"

Constraints:
    1 <= homepage.length <= 20
    1 <= url.length <= 20
    1 <= steps <= 100
    homepage and url consist of '.' or lower case English letters.
    At most 5000 calls will be made to visit, back, and forward.
"""

class BrowserHistory:
    # Dynamic array + pointer.
    #   stack       : every page ever written; slots after self.len are dead forward history
    #   curr_index  : page we're currently on
    #   len         : index of the LAST VALID page (logical end), not len(stack)
    def __init__(self, homepage: str):
        self.stack = [homepage]
        self.curr_index = 0
        self.len = 0

    def visit(self, url: str) -> None:
        # Visiting always writes to curr_index + 1 and wipes all forward history.
        #
        # e.g. stack = [yt, fb, google, yahoo], curr = 3 (yahoo), len = 3
        #   back(2)          -> curr = 1 (fb)
        #   visit(crunchy)   -> curr = 2; slot 2 already exists, so overwrite it
        #                       stack = [yt, fb, crunchy, yahoo], len = 2
        #   yahoo is still physically in the array but is past len -> dead,
        #   so forward() can't reach it, and a later visit will overwrite it.
        #
        # Overwriting dead slots instead of slicing the list (stack[:curr+1])
        # keeps visit O(1) with no reallocation.
        self.curr_index += 1
```

```
        if self.curr_index < len(self.stack):    # physical slot exists -> reuse it
            self.stack[self.curr_index] = url
        else:                                    # at the physical end -> grow
            self.stack.append(url)
        self.len = self.curr_index               # new page is now the last valid one

    def back(self, steps: int) -> str:
        # clamp at 0: can't go before the homepage
        index = max(self.curr_index - steps, 0)
        self.curr_index = index
        return self.stack[index]

    def forward(self, steps: int) -> str:
        # clamp at self.len (logical end), **NOT len(stack)**: slots past len are dead
        index = min(self.curr_index + steps, self.len)
        self.curr_index = index
        return self.stack[index]
```

```python
### design-browser-history-linked-list.py

"""
Design Browser History (NeetCode) -- another approach
Problem statement: see design-browser-history-array.py
"""

class Node:
    def __init__(self,val):
        self.val = val
        self.prev = None
        self.next = None

class BrowserHistory:

    def __init__(self, homepage: str):
        # keeps track of the latest page we r on
        self.curr = Node(homepage)

    def visit(self, url: str) -> None:
        new_node = Node(url)
        new_node.prev = self.curr
        self.curr.next = new_node
        self.curr = new_node

    def back(self, steps: int) -> str:
        # We check self.curr.prev (not self.curr) so we never
        # move onto None -- self.curr stays a valid node.
        while self.curr.prev and steps>0:
            steps -= 1
            self.curr = self.curr.prev
        return self.curr.val

    def forward(self, steps: int) -> str:
        # Same idea, forward direction. Guard on self.curr.next so
        # we never step past the newest page into None.
        while  self.curr.next and steps>0:
            steps -= 1
            self.curr = self.curr.next
        return self.curr.val


# Your BrowserHistory object will be instantiated and called as such:
# obj = BrowserHistory(homepage)
# obj.visit(url)
# param_2 = obj.back(steps)
# param_3 = obj.forward(steps)
```

```
### number-of-students-unable-to-eat-lunch.py

"""
Number of Students Unable to Eat Lunch (NeetCode)
=================================================

The school cafeteria offers circular and square sandwiches at lunch break,
referred to by numbers 0 and 1 respectively. All students stand in a queue.
Each student either prefers square or circular sandwiches.

The number of sandwiches in the cafeteria is equal to the number of students.
The sandwiches are placed in a stack. At each step:
    - If the student at the front of the queue prefers the sandwich on the top
      of the stack, they will take it and leave the queue.
    - Otherwise, they will leave it and go to the queue's end.

This continues until none of the queue students want to take the top sandwich
and are thus unable to eat.

You are given two integer arrays students and sandwiches where sandwiches[i] is
the type of the ith sandwich in the stack (i = 0 is the top of the stack) and
students[j] is the preference of the jth student in the initial queue (j = 0 is
the front of the queue). Return the number of students that are unable to eat.

Example 1:
    Input:  students = [1,1,0,0], sandwiches = [0,1,0,1]
    Output: 0

Example 2:
    Input:  students = [1,1,1,0,0,1], sandwiches = [1,0,0,0,1,1]
    Output: 3

Constraints:
    1 <= students.length, sandwiches.length <= 100
    students.length == sandwiches.length
    sandwiches[i] is 0 or 1.
    students[i] is 0 or 1.
"""

class Solution:
    def countStudents(self, students: List[int], sandwiches: List[int]) -> int:
        # using hashmap
        # ORDER OF STUDENTS IS NOT IMPORTANT
        # ORDER OF SANDWICHES IS
        student_dict = {}

        for s in students:
            if s in student_dict:
                student_dict[s] += 1
            else:
                student_dict[s] = 1

        for sandwich in sandwiches:
            if sandwich in student_dict and student_dict[sandwich]>0:
                student_dict[sandwich] -= 1
            else:
                break

        sandwich_left = student_dict.get(0,0)+student_dict.get(1,0)
        return sandwich_left
```

```
### implement-stack-using-queues-one-queue.py

"""
Implement Stack Using Queues (NeetCode)
=======================================

Implement a last-in-first-out (LIFO) stack using only two queues. The
implemented stack should support all the functions of a normal stack (push,
top, pop, and empty).

Implement the MyStack class:
    - void push(int x) Pushes element x to the top of the stack.
    - int pop() Removes the element on the top of the stack and returns it.
    - int top() Returns the element on the top of the stack.
    - boolean empty() Returns true if the stack is empty, false otherwise.

Notes:
    - You must use only standard operations of a queue, which means that only
      push to back, peek/pop from front, size and is empty operations are
      valid.
    - Depending on your language, the queue may not be supported natively. You
      may simulate a queue using a list or deque (double-ended queue) as long
      as you use only a queue's standard operations.

Example 1:
    Input:  ["MyStack", "push", "push", "top", "pop", "empty"]
            [[], [1], [2], [], [], []]
    Output: [null, null, null, 2, 2, false]

    Explanation:
    MyStack myStack = new MyStack();
    myStack.push(1);
    myStack.push(2);
    myStack.top();   // return 2
    myStack.pop();   // return 2
    myStack.empty(); // return False

Constraints:
    1 <= x <= 9
    At most 100 calls will be made to push, pop, top, and empty.
    All the calls to pop and top are valid.

Follow-up: Can you implement the stack using only one queue?
"""

class MyStack:
    # Stack using a single queue.
    # q holds the stack in order: front = top (newest).
    # push O(n), pop/top O(1)
    # Sim run (front on the left):
    #   op        after append   rotations                  q after
    #   push(1)   [1]            0                          [1]
    #   push(2)   [1, 2]         1: [2, 1]                  [2, 1]
    #   push(3)   [2, 1, 3]      2: [1, 3, 2] -> [3, 2, 1]  [3, 2, 1]
    #   pop()     returns 3, q = [2, 1]
    def __init__(self):
        self.q = deque()

    def push(self, x: int) -> None:
        self.q.append(x)                          # new element goes to the back
        for i in range(len(self.q) - 1):          # rotate the old elements (all but x)
            self.q.append(self.q.popleft())       # front -> back, until x is at the front

    def pop(self) -> int:
        return self.q.popleft()                   # front = newest

    def top(self) -> int:
        return self.q[0]

    def empty(self) -> bool:
        return len(self.q) == 0
```

```python
### implement-stack-using-queues-two-queues.py

"""
Implement Stack Using Queues (NeetCode) -- another approach
Problem statement: see implement-stack-using-queues-one-queue.py
"""

class MyStack:
    # ** MAIN OP ON PUSH **
    # Stack using two queues.
    # q1 always holds the stack in order: front = top (newest).
    # q2 is empty between calls; it's only a temp during push.
    # push O(n), pop/top O(1)
    # The idea is to use two queues: when pushing a new element,
    # we add it to the empty second queue, then move all elements
    # from the first queue behind it. This places the newest element
    # at the front, ready to be popped first.

    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x: int) -> None:
        self.q2.append(x)                          # new element goes first into empty q2
        while self.q1:                             # move old elements behind it
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1        # q2 becomes the main queue, q1 empty again

    def pop(self) -> int:
        return self.q1.popleft()                   # front = newest

    def top(self) -> int:
        return self.q1[0]

    def empty(self) -> bool:
        return len(self.q1) == 0
```

```
### design-double-ended-queue.py

"""
Design Double-ended Queue (NeetCode)
====================================

Design a Double-ended Queue class.

Your Deque class should support the following operations:
    - Deque() will initialize an empty queue.
    - bool isEmpty() will return whether the queue is empty or not.
    - void append(int value) will insert value at the end of the queue.
    - void appendleft(int value) will insert value at the beginning of the
      queue.
    - int pop() will remove and return the value at the end of the queue. If
      the queue is empty, return -1.
    - int popleft() will remove and return the value at the beginning of the
      queue. If the queue is empty, return -1.

Note: You should implement this with a linked list (or doubly linked list), not
a built-in deque.

Example 1:
    Input:  ["isEmpty", "append", 10, "isEmpty", "appendleft", 20, "popleft",
             "pop", "pop", "append", 30, "pop", "isEmpty"]
    Output: [true, null, false, null, 20, 10, -1, null, 30, true]
"""

class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class Deque:
    # Singly linked list with dummy head: dummy -> n0 -> ... -> n(size-1)
    # From dummy, k steps lands on n(k-1).
    # appendleft/popleft O(1), append/pop O(n)
    def __init__(self):
        self.head = Node(0)                 # dummy
        self.size = 0

    def isEmpty(self) -> bool:
        return (self.size == 0)

    def append(self, value: int) -> None:
        # size steps -> last node (or dummy if empty)
        curr = self.head
        for i in range(self.size):
            curr = curr.next

        new_node = Node(value)
        curr.next = new_node
        self.size += 1

    def appendleft(self, value: int) -> None:
        new_node = Node(value)
        curr = self.head
        new_node.next = curr.next           # link to old first BEFORE moving dummy
        curr.next = new_node
        self.size += 1

    def pop(self) -> int:
        if self.size == 0:
            return -1

        # size-1 steps -> second-last node (need prev to cut the link)
        curr = self.head
        for i in range(self.size - 1):
            curr = curr.next

        last_node = curr.next
        curr.next = None
        self.size -= 1
        return last_node.val

    def popleft(self) -> int:
        if self.size == 0:
```

```
            return -1

        curr = self.head
        first_node = curr.next
        curr.next = curr.next.next          # dummy skips first node
        self.size -= 1
        return first_node.val
```

```
### climbing-stairs.py

"""
Climbing Stairs (NeetCode)
==========================

You are given an integer n representing the number of steps to reach the top
of a staircase. You can climb with either 1 or 2 steps at a time.

Return the number of distinct ways to climb to the top of the staircase.

Example 1:
    Input:  n = 2
    Output: 2
    Explanation:
    1. 1 + 1 = 2
    2. 2 = 2

Example 2:
    Input:  n = 3
    Output: 3
    Explanation:
    1. 1 + 1 + 1 = 3
    2. 1 + 2 = 3
    3. 2 + 1 = 3

Constraints:
    1 <= n <= 30
"""

# ------------------------------------------------------------------------------
# PROBLEM: Climbing Stairs (LeetCode 70)
# n steps to the top; each move is 1 or 2 steps. Count distinct ways.
#
# STAIRCASE (n = 5), start on the ground at step 0:
#
#                           2-step        ___
#                         .------.   ___|   |
#                   ___  /        v|   |   |
#              ___|   |   1-step  |   |   |
#         ___|   |   |  ------>  |   |   |
#   o     |   |   |   |          |   |   |
#  /|\____|___|___|___|__________|___|___|
#    0    1    2    3           4    5
#
# DECISION TREE (brute force = O(2^n)):
#   From each step, branch to +1 or +2. Overshooting past 5 (e.g. 6) = dead end.
#
#                     0
#                   /   \
#                  1     2        <- subtree "2" appears again under 1
#                /   \   / \
#               2     3 3   4      <- repeated subtrees (3, 4, ...)
#             / \    ...
#            3   4
#           / \   \
#          4   5   5 ...
#         / \
#        5   6x
#
#   Each step's answer depends only on the two steps below it
#   -> solve each step once = O(n).
#
# DP TABLE (filled left -> right):
#   dp[i] = ways to reach step i from step 0
#
#   step:  0   1   2   3   4   5
#   dp  :  -   1   2   3   5   8      dp[i] = dp[i-1] + dp[i-2]
#                              ^ answer
#
#   ways(i) = fib(i+1)  -> same sequence as Fibonacci, offset by one index
#
#   i       :  0  1  2  3  4  5  6
#   fib(i)  :  0  1  1  2  3  5  8
#   ways(i) :  1  1  2  3  5  8  13
#
#   Same recurrence, only the starting values differ (fib: 0,1 | stairs: 1,1).
```

```python
# ------------------------------------------------------------------------

class Solution:
    def climbStairs(self, n: int) -> int:
        # Bottom-up DP (tabulation)
        # ways(i) = ways(i-1) + ways(i-2): last move was a 1-step or a 2-step
        if n<=2:
            return n            # 1 -> {1}, 2 -> {1+1, 2}

        dp = [0]*(n+1)          # n+1 so dp[n] exists; dp[0] unused
        dp[1] = 1
        dp[2] = 2

        for i in range(3,n+1):
            dp[i] = dp[i-1] + dp[i-2]

        return dp[n]            # Time: O(N), Space: O(N)

    def climbStairs_O1(self, n: int) -> int:
        # Only the last two values are ever needed -> sliding window, like fib
        one, two = 1, 1         # one = ways(i-1), two = ways(i-2)
        for _ in range(n - 1):
            one, two = one + two, one
        return one              # Time: O(N), Space: O(1)


# ------------------------------------------------------------------------------
# FIBONACCI: F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)
#   0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
# ------------------------------------------------------------------------------

def fib_recursive(n: int) -> int:
    # Plain recursion: recomputes the same subproblems (same tree as above)
    if n <= 1:
        return n                # F(0) = 0, F(1) = 1
    return fib_recursive(n-1) + fib_recursive(n-2)
    # Time: O(2^n), Space: O(n) call stack

def fib_memo(n: int, memo=None) -> int:
    # Top-down DP (memoization): same recursion, cache each F(k) once
    # memo=None, not memo={}: a mutable default would persist across calls
    if memo is None:
        memo = {}
    if n <= 1:
        return n
    if n in memo:
        return memo[n]
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]
    # Time: O(n), Space: O(n) cache + call stack

def fib_iterative(n: int) -> int:
    # Bottom-up with a 2-value sliding window: a = F(i), b = F(i+1)
    a, b = 0, 1
    while n > 0:
        a, b = b, a + b
        n -= 1
    return a
    # Time: O(n), Space: O(1)
```

```
### insertion-sort.py

"""
Insertion Sort (NeetCode)
=========================

Implement Insertion Sort.

Insertion Sort is a simple sorting algorithm that builds the sorted list one
element at a time, from left to right. It works by repeatedly taking an element
from the unsorted portion and inserting it into its correct position in the
sorted portion of the list.

Objective:
    Given a list of key-value pairs, sort the list by key using Insertion Sort.
    Return a list of lists showing the state of the array after each insertion.
    If two key-value pairs have the same key, maintain their relative order in
    the sorted list.

Input:
    pairs - a list of key-value pairs, where each key-value has an integer key
    and a string value. (0 <= pairs.length <= 100).

Example 1:
    Input:  pairs = [(5, "apple"), (2, "banana"), (9, "cherry")]
    Output: [[(5, "apple"), (2, "banana"), (9, "cherry")],
             [(2, "banana"), (5, "apple"), (9, "cherry")],
             [(2, "banana"), (5, "apple"), (9, "cherry")]]
    Note: The state of the array after the first insertion (index 0) is the
    original list.

Example 2:
    Input:  pairs = [(3, "cat"), (3, "bird"), (2, "dog")]
    Output: [[(3, "cat"), (3, "bird"), (2, "dog")],
             [(3, "cat"), (3, "bird"), (2, "dog")],
             [(2, "dog"), (3, "cat"), (3, "bird")]]
"""

# PROBLEM: Insertion Sort (NeetCode) - sort pairs by key, return list state after each insertion.
# e.g. [(5,a),(2,b),(9,c)] -> [[5a,2b,9c], [2b,5a,9c], [2b,5a,9c]]
# IDEA: pairs[0..i-1] stays sorted; swap pairs[i] left until it's in place.

class Solution:
    def insertionSort(self, pairs: List[Pair]) -> List[List[Pair]]:
        n = len(pairs)
        res = []

        for i in range(n):
            j = i-1

            # strict '>' keeps equal keys in order -> stable
            # j>=0 to make sure we dont get index out of range error
            while j>=0 and pairs[j].key > pairs[j+1].key:
                temp = pairs[j+1]
                pairs[j+1] = pairs[j]
                pairs[j] = temp
                j -= 1

            res.append(pairs[:])    # copy; appending `pairs` stores the same list n times
        return res

# Time: O(n^2) worst, O(n) if already sorted | Space: O(1) in-place
```

```
### merge-sort.py

"""
Merge Sort (NeetCode)
=====================

Implement Merge Sort.

Merge Sort is a divide-and-conquer algorithm for sorting an array or list of
elements. It works by recursively dividing the unsorted list into n sub-lists,
each containing one element. Then, it repeatedly merges sub-lists to produce
new sorted sub-lists until there is only one sub-list remaining.

Objective:
    Given a list of key-value pairs, sort the list by key using Merge Sort. If
    two key-value pairs have the same key, maintain their relative order in
    the sorted list.

Input:
    pairs - a list of key-value pairs, where each key-value has an integer key
    and a string value. (0 <= pairs.length <= 100).

Example 1:
    Input:  pairs = [(5, "apple"), (2, "banana"), (9, "cherry"), (1, "date"),
                     (9, "elderberry")]
    Output: [(1, "date"), (2, "banana"), (5, "apple"), (9, "cherry"),
             (9, "elderberry")]

Example 2:
    Input:  pairs = [(3, "cat"), (2, "dog"), (3, "bird")]
    Output: [(2, "dog"), (3, "cat"), (3, "bird")]
"""

# Definition for a pair.
# class Pair:
#     def __init__(self, key: int, value: str):
#         self.key = key
#         self.value = value

class Solution:
    def mergeSort(self, pairs: List[Pair]) -> List[Pair]:
        # Common bug: forgetting `return` here, so the caller gets None.
        return self.mergeSortHelper(pairs, 0, len(pairs) - 1)

    def mergeSortHelper(self, arr, start, end):
        # `end` is inclusive. Base case: 0 or 1 element is already sorted.
        # Without it the recursion never stops.
        if end - start + 1 <= 1:
            return arr

        # Python ints can't overflow. In Go, Java or C++ write start + (end-start)//2.
        mid = (start + end) // 2

        # Split as [start..mid] and [mid+1..end].
        # Splitting as [start..mid-1] and [mid..end] recurses forever on 2 elements.
        self.mergeSortHelper(arr, start, mid)
        self.mergeSortHelper(arr, mid + 1, end)

        self.merge(arr, start, end)
        return arr

    def merge(self, arr, start, end):
        # `mid` must be calculated exactly as in mergeSortHelper, or the halves won't line up.
        # Passing mid in as an argument is safer.
        mid = (start + end) // 2

        # Copy both halves first. Merging back into arr would otherwise overwrite
        # values not yet compared. Slice ends are exclusive, hence the +1s.
        left = arr[start:mid + 1]
        right = arr[mid + 1:end + 1]

        i = j = 0
        k = start  # Common bug: starting k at 0 overwrites the wrong part of arr.

        while i < len(left) and j < len(right):
            # Use <=, not <. On equal keys it takes from the left half first,
            # which keeps equal keys in their original order (a stable sort).
```

```
            if left[i].key <= right[j].key:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        # Copy whatever is left. Only one of these loops will run.
        # Common bug: reading arr[i] instead of left[i]. i and j are positions in
        # left and right, not in arr.
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

        return arr

# Time: O(n log n) in every case. Space: O(n) for the copied halves,
# plus O(log n) for the recursion stack.
```

```
### merge-sorted-array.py

"""
Merge Sorted Array (NeetCode)
=============================

You are given two integer arrays nums1 and nums2, both sorted in non-decreasing
order, along with two integers m and n, where:
    - m is the number of valid elements in nums1,
    - n is the number of elements in nums2.

The array nums1 has a length of (m + n), where the first m elements contain the
values to be merged, and the last n elements are set to 0 and should be
ignored.

Your task is to merge the two arrays such that the final merged array is also
sorted in non-decreasing order and stored entirely within nums1.

You must modify nums1 in-place and do not return anything from the function.

Example 1:
    Input:  nums1 = [10,20,20,40,0,0], m = 4, nums2 = [1,2], n = 2
    Output: [1,2,10,20,20,40]

Example 2:
    Input:  nums1 = [0,0], m = 0, nums2 = [1,2], n = 2
    Output: [1,2]

Constraints:
    0 <= m, n <= 200
    1 <= (m + n) <= 200
    nums1.length == (m + n)
    nums2.length == n
    -1,000,000,000 <= nums1[i], nums2[i] <= 1,000,000,000

Follow up: Can you design an algorithm that runs in O(m + n) time?
"""

class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        """
        Do not return anything, modify nums1 in-place instead.
        """
        # Fill from the back. The end of nums1 is empty space, so nothing gets overwritten.
        # Common bug: merging from the front overwrites nums1 values not yet compared.
        last_index = m + n - 1

        # m and n are counts, so subtract 1 to get the last valid index in each list.
        m -= 1
        n -= 1

        while m >= 0 and n >= 0:
            # The larger of the two goes into the current last slot.
            # On equal values nums2 goes first. That's fine for ints;
            # for objects that must stay in order (stable), use >=.
            if nums1[m] > nums2[n]:
                nums1[last_index] = nums1[m]
                m -= 1
            else:
                nums1[last_index] = nums2[n]
                n -= 1
            last_index -= 1

        # Copy whatever is left in nums2. This also covers m == 0 (nums1 started empty).
        # No loop is needed for nums1: any values left there are already in the right place.
        # Common bug: adding that loop anyway, or forgetting this one.
        while n >= 0:
            nums1[last_index] = nums2[n]
            last_index -= 1
            n -= 1

        # Common bug: returning nums1 or building a new list. The task wants nums1 changed in place
.

# Time: O(m + n). Space: O(1).
```

```
### merge-k-sorted-linked-lists.py

"""
Merge K Sorted Linked Lists (NeetCode)
======================================

You are given an array of k linked lists lists, where each list is sorted in
ascending order.

Return the sorted linked list that is the result of merging all of the
individual linked lists.

Example 1:
    Input:  lists = [[1,2,4],[1,3,5],[3,6]]
    Output: [1,1,2,3,3,4,5,6]

Example 2:
    Input:  lists = []
    Output: []

Example 3:
    Input:  lists = [[]]
    Output: []

Constraints:
    0 <= lists.length <= 1000
    0 <= lists[i].length <= 100
    -1000 <= lists[i][j] <= 1000
"""

class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        if len(lists) == 0:
            return None

        # Merge in pairs; each round halves the list count, so ~log k rounds.
        # Merging one list at a time into a running result is O(N*k).
        while len(lists) > 1:
            # Results of this round: one merged list per pair, so about half as many lists.
            # It becomes `lists` for the next round.
            # e.g. [L0, L1, L2, L3, L4] -> [L0+L1, L2+L3, L4]
            mergedLists = []

            # Use len(lists) here. It shrinks every round, so don't store it once up front.
            for i in range(0, len(lists), 2):
                l1 = lists[i]
                l2 = lists[i + 1] if i + 1 < len(lists) else None  # odd count: last list merges wi
th None
                mergedLists.append(self.mergeTwoLinkedLists(l1, l2))  # keep inside the for loop
            lists = mergedLists

        return lists[0]

    def mergeTwoLinkedLists(self, l1, l2):
        dummy = ListNode()  # anchor that never moves; the real head is dummy.next
        tail = dummy        # last node of the merged list

        while l1 and l2:
            if l1.val <= l2.val:  # <= keeps equal values in their original order
                tail.next = l1
                l1 = l1.next
            else:
                tail.next = l2
                l2 = l2.next
            tail = tail.next  # forgetting this keeps overwriting dummy.next

        tail.next = l1 or l2  # attach the remaining nodes in one step, no loop needed
        return dummy.next     # not dummy

# Time: O(N log k), where N is the total number of nodes. Space: O(k).
```

```
### quick-sort.py

"""
Quick Sort (NeetCode)
=====================

Implement Quick Sort.

Quick Sort is a divide-and-conquer algorithm. It picks a pivot element,
partitions the list so that elements smaller than the pivot are on its left and
the rest are on its right, then recursively sorts both sides.

Objective:
    Given a list of key-value pairs, sort the list by key using Quick Sort.
    Use the right-most element as the pivot. Return the sorted list.

Input:
    pairs - a list of key-value pairs, where each key-value has an integer key
    and a string value. (0 <= pairs.length <= 100).

Example 1:
    Input:  pairs = [(5, "apple"), (2, "banana"), (9, "cherry"), (1, "date"),
                     (9, "elderberry")]
    Output: [(1, "date"), (2, "banana"), (5, "apple"), (9, "elderberry"),
             (9, "cherry")]

Example 2:
    Input:  pairs = [(3, "cat"), (2, "dog"), (3, "bird")]
    Output: [(2, "dog"), (3, "bird"), (3, "cat")]

    Note: Quick Sort is NOT stable. Equal keys can swap order (cherry and
    elderberry above, cat and bird here). Merge Sort keeps them in order.

Explanation (one partition of keys [5, 2, 9, 1, 6], pivot = 6):

    left = 0, scan i = 0..3, swap arr[i] into arr[left] when arr[i] < pivot

      i=0: 5 < 6  swap(0,0), left=1    [5, 2, 9, 1, 6]
      i=1: 2 < 6  swap(1,1), left=2    [5, 2, 9, 1, 6]
      i=2: 9      skip                 [5, 2, 9, 1, 6]
      i=3: 1 < 6  swap(3,2), left=3    [5, 2, 1, 9, 6]

    put pivot at left:                 [5, 2, 1, 6, 9]
                                                ^ final position
    recurse on [5, 2, 1] and [9]

Time: O(n log n) average, O(n^2) worst (already sorted input + right-most
pivot: every partition only peels off one element). Space: O(log n) average
recursion, O(n) worst.
"""

# Definition for a pair.
# class Pair:
#     def __init__(self, key: int, value: str):
#         self.key = key
#         self.value = value

class Solution:
    def quickSort(self, pairs: List[Pair]) -> List[Pair]:
        self.quickSortHelper(pairs, 0, len(pairs) - 1)
        return pairs  # sorted in-place

    def quickSortHelper(self, arr, start, end):
        # base case: 0 or 1 element
        if end - start + 1 <= 1:
            return

        pivot = arr[end]  # pivot = rightmost
        left = start      # next slot for an element < pivot

        # move everything < pivot to the left side
        for i in range(start, end):  # excludes end (the pivot)
            if arr[i].key < pivot.key:  # strict < keeps equal keys in order (but QS still isn't st
able overall)
                temp = arr[i]
                arr[i] = arr[left]
                arr[left] = temp
```

```
                left += 1

        # put pivot in its final position
        arr[end] = arr[left]
        arr[left] = pivot

        # recurse on both sides, skipping pivot (already placed)
        self.quickSortHelper(arr, start, left - 1)
        self.quickSortHelper(arr, left + 1, end)
```

```
### sort-colors.py

"""
Sort Colors (NeetCode)
======================

You are given an array nums consisting of n elements where each element is an
integer representing a color:
    - 0 represents red
    - 1 represents white
    - 2 represents blue

Your task is to sort the array in-place such that elements of the same color
are grouped together and arranged in the order: red (0), white (1), and then
blue (2).

You must not use any built-in sorting functions to solve this problem.

Example 1:
    Input:  nums = [1,0,1,2]
    Output: [0,1,1,2]

Example 2:
    Input:  nums = [2,1,0]
    Output: [0,1,2]

Constraints:
    1 <= nums.length <= 300.
    0 <= nums[i] <= 2.

Follow up: Could you come up with a one-pass algorithm using only constant
extra space?
"""

class Solution:
    def sortColors(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        # Counting sort: values are only 0, 1, 2 -> count each
        counter = [0] * 3   # counter[v] = how many times v appears

        # Pass 1: count occurrences
        for num in nums:
            counter[num] += 1   # value doubles as index

        # Pass 2: overwrite nums in order 0s -> 1s -> 2s
        index = 0               # next write position in nums
        for i in range(3):
            while counter[i] > 0:
                nums[index] = i
                index += 1
                counter[i] -= 1
```

```
### search-2d-matrix.py

"""
Search a 2D Matrix (NeetCode)
=============================

You are given an m x n 2-D integer array matrix and an integer target.
    - Each row in matrix is sorted in non-decreasing order.
    - The first integer of every row is greater than the last integer of the
      previous row.

Return true if target exists within matrix or false otherwise.

Can you write a solution that runs in O(log(m * n)) time?

Example 1:
    Input:  matrix = [[1,2,4,8],[10,11,12,13],[14,20,30,40]], target = 10
    Output: true

Example 2:
    Input:  matrix = [[1,2,4,8],[10,11,12,13],[14,20,30,40]], target = 15
    Output: false

Constraints:
    m == matrix.length
    n == matrix[i].length
    1 <= m, n <= 100
    -10000 <= matrix[i][j], target <= 10000
"""

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        n_rows = len(matrix)
        n_cols = len(matrix[0])    # unused; can be removed

        # Step 1: binary search over ROWS to find the one that could contain target
        top = 0
        bottom = n_rows - 1

        while top <= bottom:
            mid_row = (top + bottom) // 2
            if matrix[mid_row][-1] < target:      # target bigger than row's max -> go down
                top = mid_row + 1
            elif matrix[mid_row][0] > target:     # target smaller than row's min -> go up
                bottom = mid_row - 1
            else:                                 # row[0] <= target <= row[-1] -> found the row
                break

        # loop ended without break -> no row's range contains target
        if not (top <= bottom):
            return False

        # Step 2: binary search WITHIN that row
        mid = (top + bottom) // 2    # same as mid_row (top/bottom unchanged after break)
        row = matrix[mid]
        l = 0
        r = len(row) - 1

        while l <= r:
            mid = (l + r) // 2
            if row[mid] < target:
                l = mid + 1
            elif row[mid] > target:
                r = mid - 1
            else:
                return True

        return False
```

```
### guess-number-higher-or-lower.py

"""
Guess Number Higher or Lower (NeetCode)
=======================================

We are playing the Guess Game. The game is as follows:

I pick a number from 1 to n. You have to guess which number I picked.

Every time you guess wrong, I will tell you whether the number I picked is
higher or lower than your guess.

You call a pre-defined API int guess(int num), which returns three possible
results:
    -1: Your guess is higher than the number I picked (i.e. num > pick).
     1: Your guess is lower than the number I picked (i.e. num < pick).
     0: Your guess is equal to the number I picked (i.e. num == pick).

Return the number that I picked.

Example 1:
    Input:  n = 5, pick = 3
    Output: 3

Example 2:
    Input:  n = 15, pick = 10
    Output: 10

Example 3:
    Input:  n = 1, pick = 1
    Output: 1

Constraints:
    1 <= n <= ((2^31) - 1)
    1 <= pick <= n
"""

# The guess API is already defined for you.
# @param num, your guess
# @return -1 if num is higher than the picked number
#          1 if num is lower than the picked number
#          otherwise return 0
# def guess(num: int) -> int:

class Solution:
    def guessNumber(self, n: int) -> int:
        # Classic binary search over the range 1..n
        l = 1
        r = n

        while l <= r:
            mid = (l + r) // 2
            res = guess(mid)        # (!) API returns result relative to YOUR guess

            if res == 1:            # picked > mid -> my guess too low -> go right
                l = mid + 1
            elif res == -1:         # picked < mid -> my guess too high -> go left
                r = mid - 1
            else:                   # res == 0 -> correct guess
                return mid

        return 0  # unreachable: picked number is guaranteed in 1..n
```

```python
### eating-bananas.py

"""
Koko Eating Bananas (NeetCode)
==============================

You are given an integer array piles where piles[i] is the number of bananas in
the ith pile. You are also given an integer h, which represents the number of
hours you have to eat all the bananas.

You may decide your bananas-per-hour eating rate of k. Each hour, you may
choose a pile of bananas and eat k bananas from that pile. If the pile has
less than k bananas, you may finish eating the pile but you can not eat from
another pile in the same hour.

Return the minimum integer k such that you can eat all the bananas within h
hours.

Example 1:
    Input:  piles = [1,4,3,2], h = 9
    Output: 2
    Explanation: With an eating rate of 2, you can eat the bananas in 6 hours.
    With an eating rate of 1, you would need 10 hours to eat all the bananas
    (which exceeds h=9), thus the minimum eating rate is 2.

Example 2:
    Input:  piles = [25,10,23,4], h = 4
    Output: 25

Constraints:
    1 <= piles.length <= 1,000
    piles.length <= h <= 1,000,000
    1 <= piles[i] <= 1,000,000,000
"""

class Solution:
    def minEatingSpeed(self, piles: List[int], hoursToFinishIn: int) -> int:
        # Binary search on the ANSWER (eating speed), not on the array
        leftSpeed = 1               # slowest possible speed
        rightSpeed = max(piles)     # fastest needed: every pile done in 1 hour
        result = rightSpeed            # max(piles) always works, so it's a safe default

        while leftSpeed <= rightSpeed:
            midSpeed = (leftSpeed + rightSpeed) // 2    # candidate speed

            # hours needed at this speed (one pile per hour, leftovers don't carry over)
            hours = 0
            for pile in piles:
                hours += math.ceil(pile / midSpeed)    # partial pile still costs a full hour

            if hours <= hoursToFinishIn:
                # midSpeed works -> record it, then try slower
                result = min(result, midSpeed)
                rightSpeed = midSpeed - 1
            else:
                # too slow -> need a faster speed
                leftSpeed = midSpeed + 1

        return result
```

```
### insert-into-a-binary-search-tree.py

"""
Insert into a Binary Search Tree (NeetCode)
===========================================

You are given the root node of a binary search tree (BST) and a value val to
insert into the tree. Return the root node of the BST after the insertion. It is
guaranteed that the new value does not exist in the original BST.

Note that there may exist multiple valid ways for the insertion, as long as the
tree remains a BST after insertion. You can return any of them.

Example 1:
    Input:  root = [4,2,7,1,3], val = 5
    Output: [4,2,7,1,3,5]

Example 2:
    Input:  root = [40,20,60,10,30,50,70], val = 25
    Output: [40,20,60,10,30,50,70,null,null,25]

Constraints:
    The number of nodes in the tree is in the range [0, 10^4].
    -10^8 <= Node.val <= 10^8
    All the values Node.val are unique.
    -10^8 <= val <= 10^8
    It's guaranteed that val does not exist in the original BST.
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def insertIntoBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        # Returns the new root of this subtree; the parent reattaches it.

        # Empty spot found: the new node goes here
        # (!) Must RETURN the new node. Creating it without returning means it never gets attached.
        # (!) Also covers an empty tree: the caller gets the new node back as the root.
        if root is None:
            return TreeNode(val)

        # Walk down using the BST property
        # (!) Assign the result back (root.left = ...). Just calling
        #    self.insertIntoBST(root.left, val) loses the new node when root.left is None.
        if val < root.val:
            root.left = self.insertIntoBST(root.left, val)
        elif val > root.val:
            root.right = self.insertIntoBST(root.right, val)
        # val == root.val: duplicate, so do nothing
        # (!) Check the problem's rule on duplicates; some want them sent left or right.

        # (!) Return the original root, not the new node.
        return root

# Time/space: O(h), where h = log n if balanced, n if skewed
```

```
### delete-node-in-a-bst.py

"""
Delete Node in a BST (NeetCode)
===============================

You are given a root node reference of a BST and a key, delete the node with
the given key in the BST, if present. Return the root node reference (possibly
updated) of the BST after deletion.

Basically, the deletion can be divided into two stages:
    1. Search for a node to remove.
    2. If the node is found, delete the node.

Note: There can be multiple results after deleting the node, return any one of
them.

Example 1:
    Input:  root = [5,3,9,1,4], key = 3
    Output: [5,4,9,1]
    Explanation: Another valid answer is [5,1,9,null,4].

Example 2:
    Input:  root = [5,3,6,2,4,null,7], key = 0
    Output: [5,3,6,2,4,null,7]
    Explanation: The tree does not contain a node with value 0.

Constraints:
    The number of nodes in the tree is in the range [0, 10^4].
    -10^5 <= key, Node.val <= 10^5
    All the values Node.val are unique.
    root is a valid binary search tree.
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def findMinValue(self, root):
        # Smallest value in a BST = leftmost node
        # (!) Go LEFT only. Not right, and not a full traversal.
        while root and root.left:
            root = root.left
        return root.val

    def deleteNode(self, root: Optional[TreeNode], key: int) -> Optional[TreeNode]:
        # Returns the new root of this subtree; the parent reattaches it.
        if not root:
            return None  # (!) Key not in the tree. Don't forget this base case.

        # Search: go left/right using the BST property
        # (!) Assign the result back. Deletion can change the child's root.
        if root.val < key:
            root.right = self.deleteNode(root.right, key)
        elif root.val > key:
            root.left = self.deleteNode(root.left, key)
        else:
            # Found it.
            # 0 or 1 child: replace the node with its only child (or None)
            # (!) A leaf is handled here too: root.right is None, so it returns root.left (also Non
e).
            if root.right is None:
                return root.left
            elif root.left is None:
                return root.right
            # 2 children: copy the inorder successor (min of the right subtree)
            # into this node, then delete that successor from the right subtree
            # (!) Search for the min in root.RIGHT, not in root.
            # (!) Recurse on root.right with min_val, NOT with key.
            #     key is already overwritten and doesn't exist down there.
            else:
                min_val = self.findMinValue(root.right)
                root.val = min_val
```

```
                root.right = self.deleteNode(root.right, min_val)

        return root

# Time/space: O(h), where h = log n if balanced, n if skewed
```

```
### binary-tree-from-preorder-and-inorder-traversal.py

"""
Construct Binary Tree from Preorder and Inorder Traversal (NeetCode)
====================================================================

You are given two integer arrays preorder and inorder.
    - preorder is the preorder traversal of a binary tree
    - inorder is the inorder traversal of the same tree
    - Both arrays are of the same size and consist of unique values.

Rebuild the binary tree from the preorder and inorder traversals and return its
root.

Example 1:
    Input:  preorder = [1,2,3,4], inorder = [2,1,3,4]
    Output: [1,2,3,null,null,null,4]

Example 2:
    Input:  preorder = [1], inorder = [1]
    Output: [1]

Constraints:
    1 <= inorder.length <= 1000.
    inorder.length == preorder.length
    -1000 <= preorder[i], inorder[i] <= 1000
    preorder and inorder consist of unique values.
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        # preorder helps us find the main root node
        # and then inorder helps us find the left and
        # right BT by indexing the value of the root node value
        # +++------------------------------------------------------------------+++
        # | VISUAL (Example 1)                                                 |
        # |                                                                    |
        # |             3            preorder = [3, 9, 20, 15, 7]              |
        # |           /   \          inorder  = [9, 3, 15, 20, 7]              |
        # |          9     20                                                  |
        # |               /  \                                                 |
        # |              15    7                                               |
        # |                                                                    |
        # | 1) PREORDER (Root, Left, Right): the first element is ROOT         |
        # |       [ 3 | 9 | 20, 15, 7 ]                                        |
        # |         ^   +++++++++ +++---+++---+++                                  |
        # |       root  left   right                                           |
        # |             (1)     (3)     <- sizes come from the inorder!        |
        # |                                                                    |
        # | 2) INORDER (Left, Root, Right): find the root, split there         |
        # |       [ 9 | 3 | 15, 20, 7 ]                                        |
        # |         +++++++++  ^  +++---+++---+++                                  |
        # |        left mid   right                                            |
        # |        (1)         (3)                                             |
        # |                                                                    |
        # | Preorder tells us WHO the root is.                                 |
        # | Inorder tells us HOW MANY nodes go left/right (mid = 1).           |
        # | That count lets us cut preorder too: the root, then the            |
        # | next `mid` items are the left subtree, the rest are right.         |
        # | Recurse on each side with the matching pieces of both lists.       |
        # +++------------------------------------------------------------------+++

        # (!) Use `not`, not `is None`. Slicing produces [], never None,
        #     so `is None` never fires and preorder[0] then raises IndexError.
        if not preorder or not inorder:
            return None

        # Step 1: preorder gives the root
        root = TreeNode(preorder[0])
```

```
        # Step 2: find the root in inorder. mid = index = SIZE of the left subtree.
        # (!) Works only because values are unique.
        mid = inorder.index(preorder[0])

        # Step 3: cut both lists with the same sizes and recurse
        #   preorder: [root | left (mid items) | right ]
        #              0      1 .. mid           mid+1 ..
        #   inorder:  [left (mid items) | root | right ]
        #              0 .. mid-1         mid    mid+1 ..
        # (!) The root is excluded from both sides: preorder starts at 1, and inorder skips mid.
        # (!) preorder[1:mid+1], not [1:mid]. The slice end is exclusive, and we need `mid` items.
        root.left = self.buildTree(preorder[1:mid + 1], inorder[:mid])
        root.right = self.buildTree(preorder[mid + 1:], inorder[mid + 1:])

        return root
```

```
### kth-smallest-integer-in-bst-iterative.py

"""
Kth Smallest Integer in BST (NeetCode)
======================================

Given the root of a binary search tree, and an integer k, return the kth
smallest value (1-indexed) in the tree.

A binary search tree satisfies the following constraints:
    - The left subtree of every node contains only nodes with keys less than
      the node's key.
    - The right subtree of every node contains only nodes with keys greater
      than the node's key.
    - Both the left and right subtrees are also binary search trees.

Example 1:
    Input:  root = [2,1,3], k = 1
    Output: 1

Example 2:
    Input:  root = [4,3,5,2,null], k = 4
    Output: 5

Constraints:
    1 <= k <= The number of nodes in the tree <= 1000.
    0 <= Node.val <= 1000
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        # ITERATIVE SOLN
        # Inorder traversal of a BST visits values in sorted order,
        # so the k-th node popped is the k-th smallest.
        stack = []
        curr = root
        n = 1  # 1-based count of popped nodes

        # (!) Use `or`, not `and`. The stack is empty at the start, so `and` never enters the loop.
        while curr or stack:
            # Go as far left as possible
            while curr:
                stack.append(curr)
                curr = curr.left

            curr = stack.pop()
            # (!) Count on POP (that's the visit order). k is 1-based, so n starts at 1.
            if n == k:
                return curr.val  # Early exit, no need to traverse the rest
            n += 1

            curr = curr.right

# Time: O(h + k). We go down h levels, then pop k nodes, instead of O(n) for a full traversal.
# Space: O(h) for the stack
```

```python
### kth-smallest-integer-in-bst-recursive.py

"""
Kth Smallest Integer in BST (NeetCode) -- another approach
Problem statement: see kth-smallest-integer-in-bst-iterative.py
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        # RECURSIVE SOLN
        res = []

        def inorder(node):
            if node is None:
                return

            inorder(node.left)
            res.append(node.val)
            inorder(node.right)

        inorder(root)

        return res[k-1]
```

```python
### level-order-traversal-of-binary-tree.py

"""
Binary Tree Level Order Traversal (NeetCode)
============================================

Given a binary tree root, return the level order traversal of it as a nested
list, where each sublist contains the values of nodes at a particular level in
the tree, from left to right.

Example 1:
    Input:  root = [1,2,3,4,5,6,7]
    Output: [[1],[2,3],[4,5,6,7]]

Example 2:
    Input:  root = [1]
    Output: [[1]]

Example 3:
    Input:  root = []
    Output: []

Constraints:
    0 <= The number of nodes in both trees <= 1000.
    -1000 <= Node.val <= 1000
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        # BFS level by level: the queue holds every node of the current level,
        # we process them, and their children make up the next level.

        # (!) Handle an empty tree up front, or you'd append None to the queue
        #    and crash on popped.val.
        if not root:
            return []

        # (!) Use deque, not a list. list.pop(0) is O(n), deque.popleft() is O(1).
        queue = deque()
        queue.append(root)
        # level = 0
        ans = []

        while len(queue)>0:
            # (!) KEY: snapshot the level size BEFORE the loop. Children get appended
            #    during the loop, so len(queue) keeps growing. Using it directly
            #    would mix the next level into this one.
            qlen = len(queue)
            # print(f"elements in level {level}")
            # level += 1
            levelArray = []
            for i in range(qlen):
                popped = queue.popleft()
                levelArray.append(popped.val)
                # Queue the children for the NEXT level, left before right so the order is preserve
d
                # (!) Check for None before appending, or you'll pop None later.
                if popped.left:
                    queue.append(popped.left)
                if popped.right:
                    queue.append(popped.right)

            # Exactly one level is done
            ans.append(levelArray)

        return ans

# Example:     3
#             / \
#            9   20      -> [[3], [9, 20], [15, 7]]
```

```
#               /   \
#              15    7
#
# Time: O(n), every node is enqueued and dequeued once
# Space: O(w), where w = max width. That's up to ~n/2 for the last level of a full tree.
```

```
### binary-tree-right-side-view.py

"""
Binary Tree Right Side View (NeetCode)
======================================

You are given the root of a binary tree. Return only the values of the nodes
that are visible from the right side of the tree, ordered from top to bottom.

Example 1:
    Input:  root = [1,2,3]
    Output: [1,3]

Example 2:
    Input:  root = [1,2,3,4,5,6,7]
    Output: [1,3,7]

Constraints:
    0 <= number of nodes in the tree <= 100
    -100 <= Node.val <= 100
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        # BFS

        queue = deque()
        ans = []
        queue.append(root)

        while len(queue)>0:
            qlen = len(queue)
            rightSide = None
            for _ in range(qlen):
                popped = queue.popleft()
                if popped:
                    rightSide = popped
                    if popped.left:
                        queue.append(popped.left)
                    if popped.right:
                        queue.append(popped.right)

            # when the for loop finishes over the qlen rightSide will have the rightmost value unle
ss None
            if rightSide:
                ans.append(rightSide.val)

        return ans
```

```
### cheatsheet.py

"""
BACKTRACKING: GENERIC LOGIC

Core idea: CHOOSE -> EXPLORE -> UN-CHOOSE
  Build a candidate step by step. If a branch can't lead to a goal, undo the
  last choice and try the next option.

Three problem types:
  | Type                | Explore step                          | Example                    |
  |---------------------|---------------------------------------|----------------------------|
  | Find ONE            | if recurse(): return True             | canReachLeaf, hasPathSum   |
  | Find ALL            | recurse() and collect path[:]         | Path Sum II, subsets       |
  | Count / optimise    | total += recurse() / min(...)         | number of paths            |

When do you need pop() (un-choose)?
  - SHARED MUTABLE state (list, set, visited grid) -> YES, undo the change.
  - IMMUTABLE value passed as an argument (int sum, string) -> NO.
    It can't be changed in place: `sum -= x` makes a NEW local value and the
    caller's value stays as it was, so there's nothing to undo.
  - A NEW list passed down each call, e.g. dfs(child, path + [node.val]) -> NO.
    Each call gets its own list, so nothing is shared. Simpler, but it costs
    an O(h) copy per call.

(!) Common mistakes:
  1. `return recurse(left)` alone skips the right side. Use `if recurse(left): return True`,
     or `return recurse(left) or recurse(right)` (`or` only tries right if left fails).
  2. res.append(path) stores a REFERENCE, which is empty after all the pops.
     Use path[:] or list(path).
  3. Forgetting pop() on shared state leaks the path between sibling branches.
  4. Double-counting: if you already did `remaining -= node.val`, don't subtract it again in the ca
ll.
  5. Leaf check: decide whether you compare BEFORE or AFTER using the current node, and stay consis
tent.
"""


# ---------------------------------------------------------------------------
# GENERIC TEMPLATE (pseudocode: invalid, is_goal, options, next_state,
# record, current and path are placeholders, so this doesn't run as-is)
# ---------------------------------------------------------------------------
def backtrack(state):
    # 1. DEAD END: invalid or out of bounds
    if invalid(state):
        return False

    # 2. CHOOSE: add the current element to the shared path
    path.append(current)

    # 3. GOAL CHECK
    if is_goal(state):
        record(path[:])       # or `return True` for find-one problems

    # 4. EXPLORE: try every option
    for option in options(state):
        if backtrack(next_state(option)):
            return True       # find-one only; drop this line for find-all

    # 5. UN-CHOOSE: undo step 2 so siblings start clean
    path.pop()
    return False


# ---------------------------------------------------------------------------
# NeetCode: Backtracking (Trees)
# Problem: Is there a root-to-leaf path that contains NO zeros?
#
#          4
#         / \
#        0   1          canReachLeaf -> True  (4 -> 1 -> 2)
#             \         leafPath     -> path = [4, 1, 2]
#              2
#
# canReachLeaf answers YES/NO. leafPath also returns WHICH path.
# ---------------------------------------------------------------------------
class TreeNode:
```

```
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None


# 0. FIND ONE, no state at all -> no pop()
#    Achieves: True if any root-to-leaf path avoids every 0-valued node.
#    A 0 node acts as a wall: that branch is a dead end.
def canReachLeaf(root):
    if not root or root.val == 0:              # 1. dead end: no node, or a blocked node
        return False

    if not root.left and not root.right:       # 3. goal: a leaf reached without hitting a 0
        return True
    if canReachLeaf(root.left):                # 4. explore left, stop on success
        return True
    if canReachLeaf(root.right):               #    left failed -> try right
        return True
    return False                               # both sides blocked
    # No choose/un-choose steps: nothing is tracked, so there's nothing to undo.


# 1. FIND ONE + shared state -> needs pop()
#    Achieves: same check as canReachLeaf, but `path` ends up holding the
#    actual valid path (e.g. [4, 1, 2]). If no path exists, `path` is left empty.
def leafPath(root, path):
    if not root or root.val == 0:              # 1. dead end
        return False
    path.append(root.val)                      # 2. choose: tentatively add this node

    if not root.left and not root.right:       # 3. goal: reached a leaf
        return True                            #    (!) no pop: keep the path, it's the answer
    if leafPath(root.left, path):              # 4. explore left
        return True
    if leafPath(root.right, path):             #    explore right
        return True

    path.pop()                                 # 5. un-choose: this node led nowhere,
    return False                               #    remove it before the parent tries its other chi
ld


# ------------------------------------------------------------------------
# 2. FIND ONE + immutable state -> no pop()
#    LeetCode 112: does any root-to-leaf path sum to targetSum?
# ------------------------------------------------------------------------
class Solution:
    def hasPathSum(self, root, targetSum: int) -> bool:
        if not root:                                        # 1. dead end
            return False

        targetSum -= root.val                               # 2. choose (local int copy)

        if not root.left and not root.right:                # 3. goal
            return targetSum == 0                           # (!) compared AFTER subtracting

        if self.hasPathSum(root.left, targetSum):           # 4. explore
            return True                                     # (!) don't `return` the left result direct
ly
        if self.hasPathSum(root.right, targetSum):
            return True

        return False                                        # 5. nothing to undo: int is passed by valu
e


# ------------------------------------------------------------------------
# 3. FIND ALL + shared state -> needs pop() AND copy on record
#    LeetCode 113: return every root-to-leaf path that sums to targetSum.
# ------------------------------------------------------------------------
def pathSum(root, targetSum):
    res, path = [], []

    def dfs(node, remaining):
        if not node:                                        # 1. dead end
            return
```

```
        path.append(node.val)                           # 2. choose
        remaining -= node.val

        if not node.left and not node.right and remaining == 0:
            res.append(path[:])                         # 3. goal (!) COPY, `path` keeps changing

        dfs(node.left, remaining)                       # 4. explore both, no early exit
        dfs(node.right, remaining)

        path.pop()                                      # 5. un-choose

    dfs(root, targetSum)
    return res


# Time: O(n) for find-one/count on a tree.
#       O(n * h) for find-all on a tree (copying each path costs O(h)).
# Space: O(h) recursion + O(h) path
```

```python
### path-sum.py

"""
Path Sum (NeetCode)
===================

You are given the root of a binary tree and an integer targetSum, return true
if the tree has a root-to-leaf path such that adding up all the values along
the path equals targetSum.

A leaf is a node with no children.

Example 1:
    Input:  root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22
    Output: true
    Explanation: The root-to-leaf path 5 -> 4 -> 11 -> 2 sums to 22.

Example 2:
    Input:  root = [1,2,3], targetSum = 5
    Output: false
    Explanation: The root-to-leaf paths are 1 -> 2 (sum 3) and 1 -> 3 (sum 4).

Example 3:
    Input:  root = [], targetSum = 0
    Output: false
    Explanation: An empty tree has no root-to-leaf paths.

Constraints:
    The number of nodes in the tree is in the range [0, 5000].
    -1000 <= Node.val <= 1000
    -1000 <= targetSum <= 1000
"""

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def hasPathSum(self, root: Optional[TreeNode], targetSum: int) -> bool:
        if not root:                # (!) empty tree -> False, even if targetSum == 0
            return False

        # int is local to this call -> nothing to undo (no pop)
        # (!) subtract only here, not again in the recursive calls
        targetSum = targetSum - root.val

        # (!) goal only at a LEAF; [1,2], target 1 -> False (1 isn't a leaf)
        if not root.left and not root.right and targetSum==0:
            return True

        # (!) not `return self.hasPathSum(root.left, ...)`: a False would skip the right side
        if self.hasPathSum(root.left, targetSum):
            return True
        if self.hasPathSum(root.right, targetSum):
            return True

        return False

# (!) values can be negative: don't stop early when targetSum < 0
# Time: O(n), Space: O(h)
```

```
### subsets-include-exclude.py

r"""
Subsets (NeetCode)
==================

Given an array nums of unique integers, return all possible subsets of nums.

The solution set must not contain duplicate subsets. You may return the
solution in any order.

Example 1:
    Input:  nums = [1,2,3]
    Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Example 2:
    Input:  nums = [7]
    Output: [[],[7]]

Constraints:
    1 <= nums.length <= 10
    -10 <= nums[i] <= 10

Explanation (decision tree for nums = [1,2,3]):

    Screenshot: subsets-include-exclude.png (in this folder)

    Each level decides ONE element: include it (+n) or skip it.

                                []
                +1 /                        \ skip 1
                [1]                          []
          +2 /          \ skip          +2 /          \ skip
          [1,2]           [1]             [2]           []
     +3 /    \ skip   +3 /  \ skip    +3 /  \ skip   +3 /  \ skip
  [1,2,3]   [1,2]    [1,3]    [1]    [2,3]    [2]     [3]     []

    Nodes per level: 1 -> 2 -> 4 -> 8. Every level doubles, so n elements
    give 2^n leaves, and every leaf is one subset (including []).

    In the code:
      - left branch  = subsets.append(nums[index]); dfs(index+1)
      - right branch = subsets.pop();               dfs(index+1)
      - a leaf       = index == len(nums) -> record a copy of subsets
"""

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        res = []
        subsets = []                # current subset, shared by all calls

        def dfs(index):
            # decided include/exclude for every element -> one full subset
            if index>=len(nums):
                # (!) copy: subsets keeps changing after this
                # (!) append() returns None; works, but `append` then `return` is clearer
                return res.append(subsets[:])

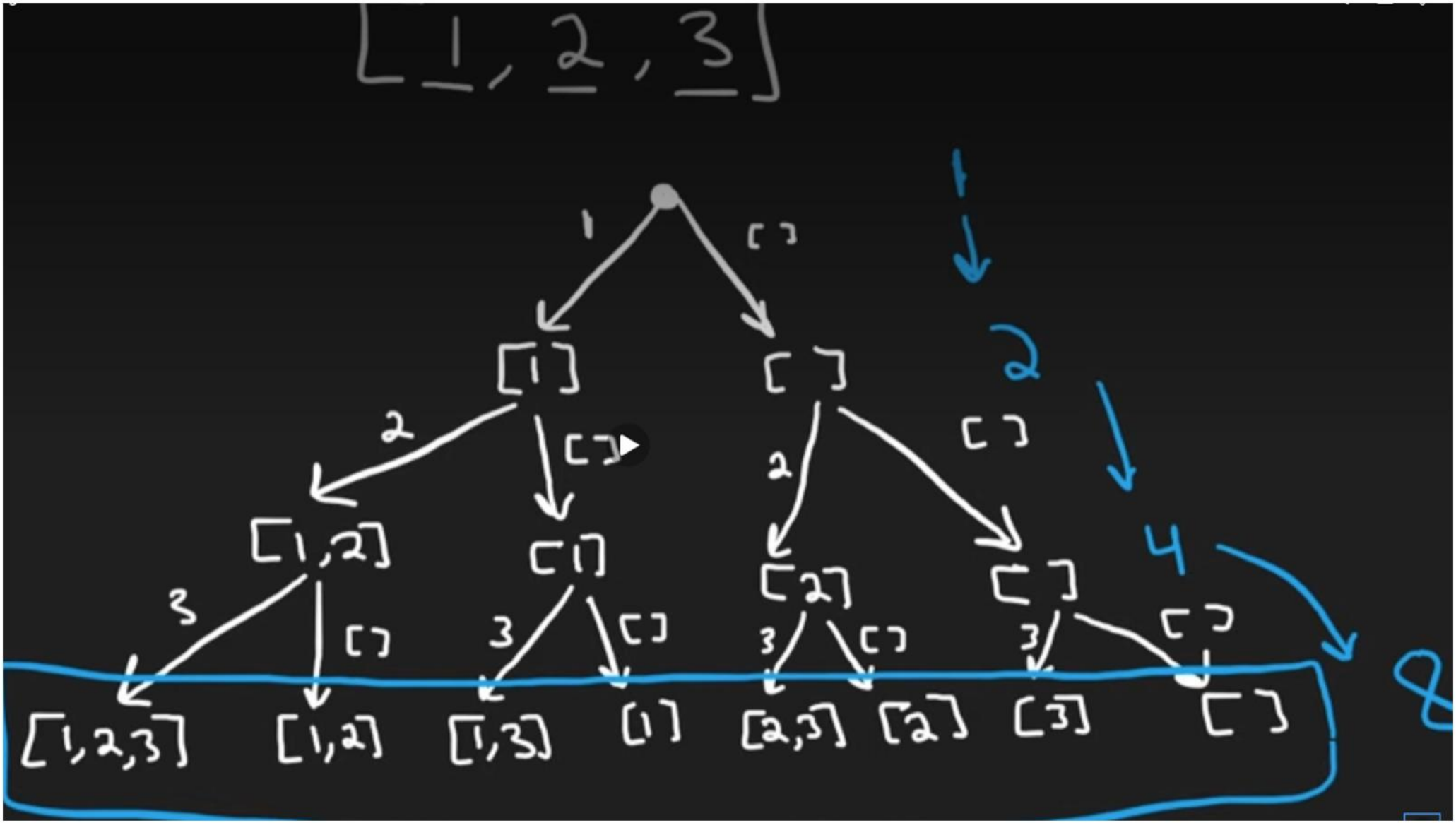
            # CHOOSE: include nums[index], then EXPLORE
            subsets.append(nums[index])
            dfs(index+1)

            # UN-CHOOSE: (!) pop before the exclude branch, or nums[index] leaks into it
            subsets.pop()
            dfs(index+1)            # EXPLORE without nums[index]

        dfs(0)
        return res

# Time: O(n * 2^n), Space: O(n)
```

Diagram: Data Structures & Algorithms/07 Backtracking/1 Tree Maze/subsets/subsets-include-exclude.png



```python
### subsets-loop.py

"""
Subsets (NeetCode) -- another approach
Problem statement: see subsets-include-exclude.py
"""

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        res = []
        self.backtrack(nums, 0, [], res)
        return res

    def backtrack(self, nums, start, path, res):
        res.append(path[:])                 # every node is a subset; (!) copy

        for j in range(start, len(nums)):
            path.append(nums[j])            # CHOOSE
            self.backtrack(nums, j + 1, path, res)    # EXPLORE ((!) j + 1, not start + 1)
            path.pop()                      # UN-CHOOSE
```

```
### combination-target-sum.py

r"""
Combination Target Sum (NeetCode)
=================================

You are given an array of distinct integers nums and a target integer target.
Your task is to return a list of all unique combinations of nums where the
chosen numbers sum to target.

The same number may be chosen from nums an unlimited number of times. Two
combinations are the same if the frequency of each of the chosen numbers is
the same, otherwise they are different.

You may return the combinations in any order and the order of the numbers in
each combination can be in any order.

Example 1:
    Input:  nums = [2,5,6,9], target = 9
    Output: [[2,2,5],[9]]
    Explanation:
    2 + 2 + 5 = 9. We use 2 twice, and 5 once.
    9 = 9. We use 9 once.

Example 2:
    Input:  nums = [3,4,5], target = 16
    Output: [[3,3,3,3,4],[3,3,5,5],[4,4,4,4],[3,4,4,5]]

Example 3:
    Input:  nums = [3], target = 5
    Output: []

Constraints:
    All elements of nums are distinct.
    1 <= nums.length <= 20
    2 <= nums[i] <= 30
    2 <= target <= 30

Explanation (decision tree for nums = [2,3,6,7], target = 7):

    Screenshot: combination-target-sum.png (in this folder)

    At every node there are two choices for nums[i]:
      +n     = include nums[i] and STAY at i  (it can be used again)
      skip n = never use nums[i] again, move to i+1

                                  []
                  +2 /                              \ skip 2
                   [2]                               []
          +2 /              \ skip 2       +3 /             \ skip 3
         [2,2]                [2]             [3]              []
    +2 /      \ skip 2   +3 /    \ skip 3    X          +6 /      \ skip 6
  [2,2,2]     [2,2]     [2,3]      [2]                  [6]         []
     X     +3 /  \ skip  X          X                    X    +7 /    \ skip 7
         [2,2,3]  [2,2]                                      [7]      []
          = 7       X                                        = 7      end
         FOUND                                              FOUND

    X = every path below goes over 7 (or runs out of numbers), so it's cut off

    Answer: [[2,2,3], [7]]

    Why no duplicates like [3,2,2]: once you go down "skip 2", 2 is gone for
    that whole subtree. So every combination is built in the order of nums,
    and each one is found exactly once.

    Why stop at total > target: nums are all positive, so adding more can
    only make the total bigger. That branch can never come back to 7.

    In the code:
      - "+n" branch    = subsets.append(nums[index]); dfs(index, total + nums[index])
      - "skip n" branch = subsets.pop();               dfs(index + 1, total)
      - FOUND          = total == target -> record a copy of subsets
      - X              = total > target or index past the end -> return
"""

class Solution:
```

```
    def combinationSum(self, nums: List[int], target: int) -> List[List[int]]:
        result = []
        subsets = []                # current combination, shared by all calls

        def dfs(index, total):
            # (!) goal check before bounds, or a combo ending on the last index is lost
            if total==target:
                return result.append(subsets[:])    # (!) copy, subsets keeps changing

            # out of numbers, or overshot (nums are positive, so total only grows)
            if index >= len(nums) or total > target:
                return

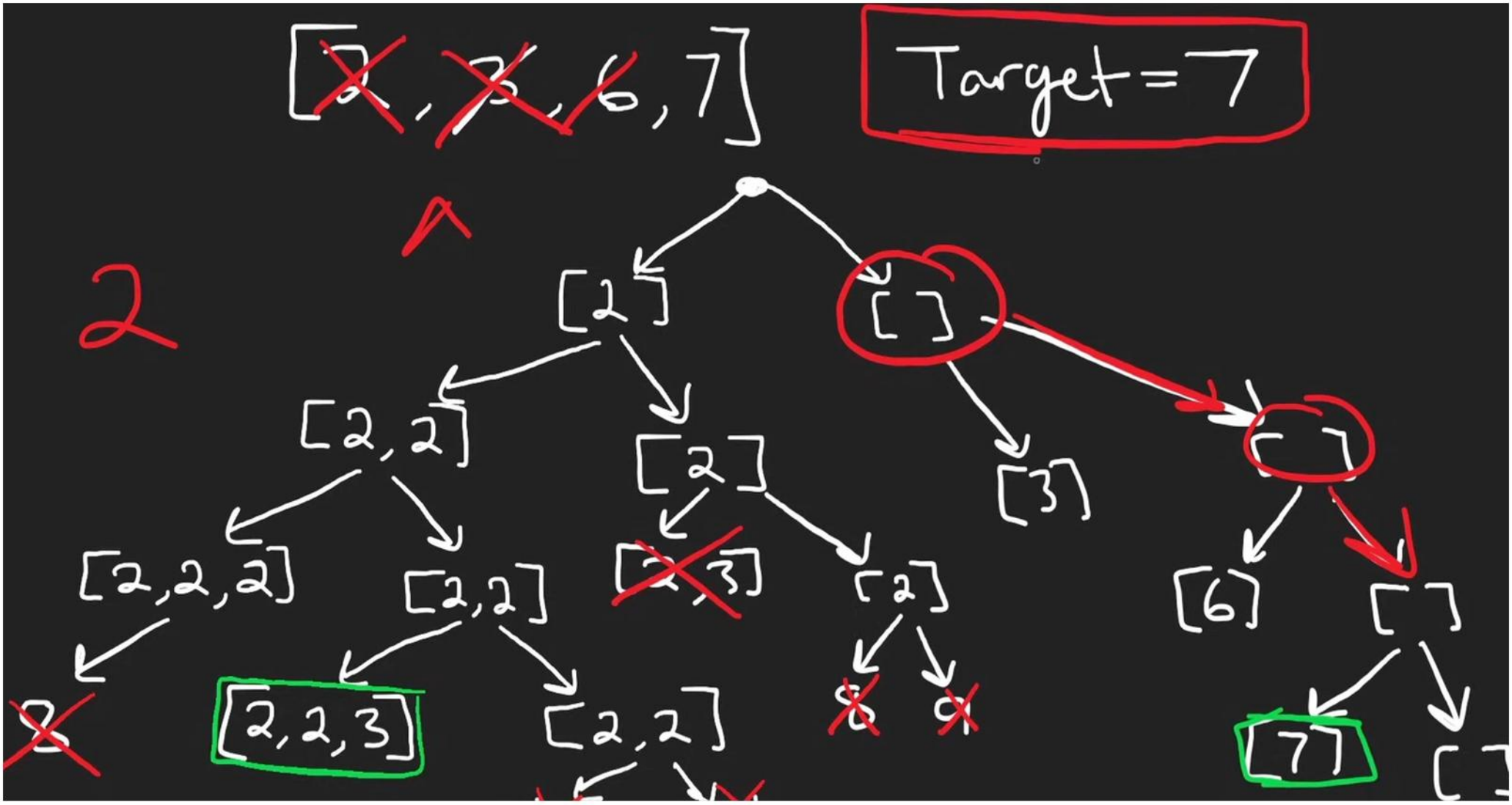
            # CHOOSE nums[index]; (!) stay at index since reuse is allowed
            subsets.append(nums[index])
            dfs(index, total+nums[index])

            # UN-CHOOSE, then skip nums[index] for good
            subsets.pop()
            dfs(index+1, total)     # (!) total, not total+nums[index]: it was popped

        dfs(0,0)
        return result

# Time: O(2^(target/min(nums))), Space: O(target/min(nums))
```

Diagram: Data Structures & Algorithms/07 Backtracking/1 Tree Maze/combination-target-sum/combination-target-sum.png



```
### k-closest-points-to-origin.py

"""
K Closest Points to Origin (NeetCode)
=====================================

You are given an 2-D array points where points[i] = [xi, yi] represents the
coordinates of a point on an X-Y axis plane. You are also given an integer k.

Return the k closest points to the origin (0, 0).

The distance between two points is defined as the Euclidean distance
(sqrt((x1 - x2)^2 + (y1 - y2)^2)).

You may return the answer in any order.

Example 1:
    Input:  points = [[0,2],[2,2]], k = 1
    Output: [[0,2]]
    Explanation: The distance between (0, 2) and the origin (0, 0) is 2.
    The distance between (2, 2) and the origin is sqrt(2^2 + 2^2) = 2.82842.
    So the closest point to the origin is (0, 2).

Example 2:
    Input:  points = [[0,2],[2,0],[2,2]], k = 2
    Output: [[0,2],[2,0]]
    Explanation: The output [[2,0],[0,2]] would also be accepted.

Constraints:
    1 <= k <= points.length <= 1000
    -100 <= points[i][0], points[i][1] <= 100
"""

class Solution:
    def kClosest(self, points: List[List[int]], k: int) -> List[List[int]]:
        res = []

        min_heap = []
        for x, y in points:
            # skip sqrt: comparing x^2+y^2 gives the same order as true distance
            dist = (x**2) + (y**2)
            # tuple: heap orders by first element (dist), then x, y for ties
            min_heap.append((dist, x, y))

        # * heapq.heapify(list) -> turns list into a min-heap IN-PLACE, O(n)
        heapq.heapify(min_heap)

        while k > 0:
            # * heapq.heappop(heap) -> removes & returns smallest, O(log n)
            dist, x, y = heapq.heappop(min_heap)
            res.append([x, y])
            k -= 1

        return res
```

```
### two-integer-sum.py

"""
Two Integer Sum (NeetCode)
==========================

Given an array of integers nums and an integer target, return the indices i and
j such that nums[i] + nums[j] == target and i != j.

You may assume that every input has exactly one pair of indices i and j that
satisfy the condition.

Return the answer with the smaller index first.

Example 1:
    Input:  nums = [3,4,5,6], target = 7
    Output: [0,1]
    Explanation: nums[0] + nums[1] == 7, so we return [0, 1].

Example 2:
    Input:  nums = [4,5,6], target = 10
    Output: [0,2]

Example 3:
    Input:  nums = [5,5], target = 10
    Output: [0,1]

Constraints:
    2 <= nums.length <= 1000
    -10,000,000 <= nums[i] <= 10,000,000
    -10,000,000 <= target <= 10,000,000
    Only one valid answer exists.
"""

class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        num_index_map = {}
        for idx, num in enumerate(nums):
            num_index_map[num] = idx

        for idx, num in enumerate(nums):
            diff = target - num

            if diff in num_index_map and num_index_map[diff]!=idx:
                return [idx,num_index_map[diff]]

        return []
```

```
### lru-cache.py

"""
LRU Cache (NeetCode)
====================

Implement the Least Recently Used (LRU) cache class LRUCache:
    - LRUCache(int capacity) Initialize the LRU cache of size capacity.
    - int get(int key) Return the value corresponding to the key if the key
      exists, otherwise return -1.
    - void put(int key, int value) Update the value of the key if the key
      exists. Otherwise, add the key-value pair to the cache. If the
      introduction of the new pair causes the cache to exceed its capacity,
      remove the least recently used key.

A key is considered used if a get or a put operation is called on it.

Ensure that get and put each run in O(1) average time complexity.

Example 1:
    Input:
    ["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
    [[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

    Output:
    [null, null, null, 1, null, -1, null, -1, 3, 4]

Constraints:
    1 <= capacity <= 100
    0 <= key <= 1000
    0 <= value <= 1000

Explanation (state after put(3,3) in Example 1, capacity = 2):

    Screenshot: lru-cache.png (in this folder)

    Hash map gives O(1) lookup, doubly linked list gives O(1) reorder/evict.

                    LRU     most recent
                     |           |
        left  <->  [1,1]  <->  [3,3]  <->  right
                     ^           ^
    cache = {        |           |
        1: ----------+           |
        3: ----------------------+
    }

    Node = | key, val |
           | prev | next |

    How it got here:
      put(1,1), put(2,2)  ->  left <-> [1,1] <-> [2,2] <-> right
      get(1)              ->  1 is used, moves to the right end:
                              left <-> [2,2] <-> [1,1] <-> right
      put(3,3)            ->  over capacity, evict left.next = [2,2]:
                              left <-> [1,1] <-> [3,3] <-> right
                              and delete key 2 from cache

    Why store key in Node: on eviction we only have the node (left.next),
    and need its key to delete it from cache.

    Why left/right dummy nodes: insert and remove never have to handle an
    empty list or a missing neighbour.
"""

class Node:
    def __init__(self, key, val):
        self.key = key              # (!) store the key too: eviction needs it to delete from cache
        self.val = val
        self.prev = None
        self.next = None

class LRUCache:
    # cache: key -> Node, for O(1) lookup
    # list:  left <-> ... <-> right   (left.next = LRU, right.prev = most recent)
    def __init__(self, capacity: int):
        self.cache = {}
```

```
        # dummy sentinels, so insert/remove never need edge cases for an empty list
        self.left,self.right = Node(0,0), Node(0,0)
        self.left.next = self.right
        self.right.prev = self.left
        self.capacity = capacity

    def remove(self, node):
        # unlink node: its neighbours point at each other
        prev, nxt = node.prev, node.next
        prev.next = nxt
        nxt.prev = prev

    def insert(self, node):
        # insert just before right = most recent
        prev, nxt = self.right.prev, self.right
        prev.next = node
        nxt.prev = node
        node.prev = prev
        node.next = nxt

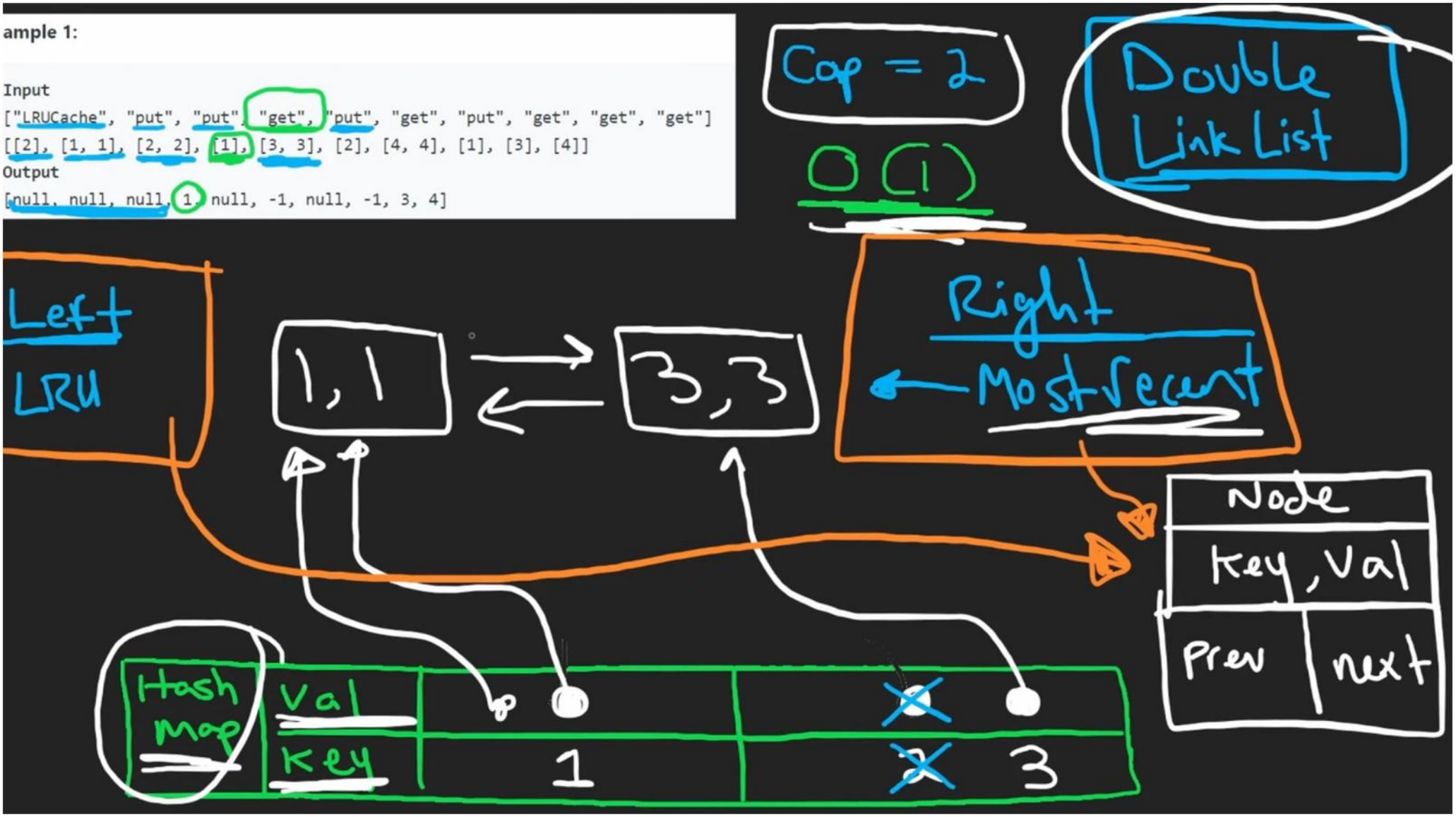
    def get(self, key: int) -> int:
        if key in self.cache:
            # (!) a read counts as a use: move it to the most recent end
            self.remove(self.cache[key])
            self.insert(self.cache[key])
            return self.cache[key].val
        return -1

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            # (!) unlink the old node first, or it stays in the list as a ghost
            self.remove(self.cache[key])
        self.cache[key] = Node(key,value)
        self.insert(self.cache[key])

        # (!) evict AFTER inserting; `>` because we're one over capacity now
        if len(self.cache) > self.capacity:
            # (!) save the node first: remove() changes what left.next points to
            lru = self.left.next
            self.remove(lru)
            del self.cache[lru.key]

# Time: O(1) for get and put. Space: O(capacity)
```

Diagram: Data Structures & Algorithms/09 Hashing/1 Hash Usage/lru-cache/lru-cache.png



```
### cheatsheet.py

"""
GRAPHS: MATRIX DFS / BFS CHEATSHEET

1. Two kinds of matrix DFS. The only difference in code is whether you undo the visit.

   |                | Reachability / flood fill     | Path counting / finding           |
   |----------------|-------------------------------|-----------------------------------|
   | Question       | Which cells can I reach?      | How many ways (or which ways)?    |
   | `visit` means  | visited ever                  | on the CURRENT path               |
   | Undo?          | No, stays visited             | Yes, visit.remove() (backtrack)   |
   | Each cell      | processed once                | once per path through it          |
   | Time           | O(rows * cols)                | O(4^(rows * cols))                |
   | Examples       | Number of Islands, Max Area   | Count Paths (matrixDFS),          |
   |                | of Island, Rotting Oranges    | Word Search                       |

  How to tell: should two different routes to the same cell count separately?
    - No  -> mark once, never unmark. Plain DFS (or BFS).
    - Yes -> free the cell when you back out of it. Backtracking
             (see the Backtracking cheatsheet).

  If you may only move right/down, paths can't loop back: no `visit` needed, and
  many paths reach the same cell, so cache each cell's result. That's DP
  (Unique Paths): O(rows * cols) instead of exponential.

2. BFS vs DFS with a deque: just which end you pop from.

    row, col = q.popleft()   # queue (FIFO): BFS, spreads out ring by ring
    row, col = q.pop()       # stack (LIFO): DFS, follows one direction deep first

  For reachability (Number of Islands) both give the same answer and O(rows * cols).
  Only BFS gives shortest paths in an unweighted grid (Shortest Path in Binary
  Matrix, Rotting Oranges), because it reaches cells in order of distance.

3. Mark visited when you PUSH, not when you pop.
    - Marking on pop lets the same cell be pushed many times by different
      neighbours. Still correct, but slower and uses more memory.
    - With mark-on-push, the stack version isn't exactly recursive DFS order
      (the first neighbour to push a cell claims it). Fine for reachability; use
      real recursion when the order matters (cycle detection, topological sort).

4. Recursion depth.
    Recursive DFS on a big all-land grid (300 x 300 = 90,000 cells) goes ~90,000
    calls deep. Python's default limit is ~1,000, so it crashes. Use a stack
    (q.pop()) or BFS for large grids.

(!) Common mistakes:
  1. Bounds: valid indices are 0..rows-1, so check r == rows (or r >= rows), not r > rows.
  2. Check bounds BEFORE grid[r][c], or a negative index silently wraps around in Python.
  3. Forgetting visit.remove() in path counting -> undercounts.
     Adding visit.remove() in flood fill -> exponential time.
  4. Counting islands: start a new search only from unvisited land, then islands += 1.
"""

from collections import deque

DIRS = [(1, 0), (-1, 0), (0, 1), (0, -1)]       # down, up, right, left


# ---------------------------------------------------------------------------
# FLOOD FILL (reachability): visit stays marked. O(rows * cols)
# ---------------------------------------------------------------------------
def numIslands(grid):
    rows, cols = len(grid), len(grid[0])
    visit = set()

    def explore(r, c):
        q = deque([(r, c)])
        visit.add((r, c))                           # mark on push
        while q:
            row, col = q.popleft()                  # popleft = BFS, pop = DFS; same count
            for dr, dc in DIRS:
                nr, nc = row + dr, col + dc
                if (0 <= nr < rows and 0 <= nc < cols
                        and grid[nr][nc] == "1" and (nr, nc) not in visit):
                    visit.add((nr, nc))
```

```
                    q.append((nr, nc))

    islands = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1" and (r, c) not in visit:
                explore(r, c)                    # marks the whole island
                islands += 1
    return islands


# ------------------------------------------------------------------------
# PATH COUNTING (backtracking): visit is undone on the way back. O(4^(rows*cols))
# ------------------------------------------------------------------------
def countPaths(grid):
    rows, cols = len(grid), len(grid[0])
    visit = set()                                # cells on the CURRENT path

    def dfs(r, c):
        if (min(r, c) < 0 or r == rows or c == cols
                or (r, c) in visit or grid[r][c] == 1):
            return 0
        if r == rows - 1 and c == cols - 1:
            return 1

        visit.add((r, c))                        # CHOOSE
        count = sum(dfs(r + dr, c + dc) for dr, dc in DIRS)    # EXPLORE
        visit.remove((r, c))                     # UN-CHOOSE
        return count

    return dfs(0, 0)
```

```
### matrix-dfs.py

"""
Matrix Depth-First Search: Count Paths (NeetCode)
=================================================

You are given a binary matrix grid of size ROWS x COLS, where 0 is an open cell
and 1 is a blocked cell.

Return the number of unique paths from the top-left cell (0, 0) to the
bottom-right cell (ROWS - 1, COLS - 1).

    - You may move up, down, left or right.
    - A path may only move along 0's (it can't step on a 1).
    - A path can't visit the same cell more than once.

Example 1:
    Input:  grid = [[0,0,0,0],
                    [1,1,0,0],
                    [0,0,0,1],
                    [0,1,0,0]]
    Output: 2

Example 2:
    Input:  grid = [[0,0],
                    [0,0]]
    Output: 2
    Explanation: right -> down, or down -> right.

Example 3:
    Input:  grid = [[0,1],
                    [1,0]]
    Output: 0
    Explanation: Every route to the bottom-right is blocked.

Concepts (two kinds of matrix DFS, BFS vs DFS): see the Graphs cheatsheet.
"""

class Solution:
    def countPaths(self, grid: List[List[int]]) -> int:
        nRows, nCols = len(grid),len(grid[0])
        visit = set()               # cells on the CURRENT path, shared by all calls

        def dfs(r,c):
            # dead end: off the grid, already on this path, or a wall (1)
            # (!) == nRows / == nCols, since valid indices are 0..n-1
            if (r<0 or r==nRows
                or c<0 or c==nCols
                or (r,c)in visit
                or grid[r][c]==1):
                return 0

            # reached bottom-right: this path counts once
            # (!) return before visit.add, so there's nothing to undo here
            if r==nRows-1 and c==nCols-1:
                return 1

            visit.add((r,c))        # CHOOSE: put this cell on the path

            count = 0
            dirs = [(0,1),(1,0),(0,-1),(-1,0)]   # right, down, left, up

            # EXPLORE: add up the paths from every neighbour (count all, no early return)
            for dr, dc in dirs:
                row, col = r+dr, c+dc
                count += dfs(row,col)

            # UN-CHOOSE: (!) other paths may pass through this cell, so free it again
            # (a permanent visited set, as in Number of Islands, would undercount)
            visit.remove((r,c))

            return count

        return dfs(0,0)

# Time: O(4^(rows*cols)) worst case, every path is explored
# Space: O(rows*cols) for the recursion and visit
```

```
### count-number-of-islands-dfs.py

"""
Count Number of Islands (NeetCode)
==================================

Given a 2D grid grid where '1' represents land and '0' represents water, count
and return the number of islands.

An island is formed by connecting adjacent lands horizontally or vertically
and is surrounded by water. You may assume water is surrounding the grid
(i.e., all the edges are water).

Example 1:
    Input: grid = [
        ["0","1","1","1","0"],
        ["0","1","0","1","0"],
        ["1","1","0","0","0"],
        ["0","0","0","0","0"]
      ]
    Output: 1

Example 2:
    Input: grid = [
        ["1","1","0","0","1"],
        ["1","1","0","0","1"],
        ["0","0","1","0","0"],
        ["0","0","0","1","1"]
      ]
    Output: 4

Constraints:
    1 <= grid.length, grid[i].length <= 100
    grid[i][j] is '0' or '1'.

Explanation (what the search does):

    The outer loop finds an island by hitting any single 1. DFS/BFS then
    spreads to all the land connected to it and marks it, so the loop skips
    the rest of that island. Each island is counted exactly once.

      grid:      loop hits (0,0) -> marks the island -> islands = 1
      1 1 0 0    x x 0 0
      1 0 0 1    x 0 0 1     (0,1), (1,0) already marked: skipped
      0 0 1 1    0 0 1 1
                 loop hits (1,3) -> marks it + (2,2), (2,3) -> islands = 2

    Without the spreading step every 1 would count: 6 instead of 2.

    DFS (recursion) or BFS (queue) both work: this is reachability, so the
    order cells are visited in doesn't matter. See the Graphs cheatsheet.
"""

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        nRows, nCols = len(grid), len(grid[0])
        visit = set()                       # cells already part of a counted island
        directions = [(0,1), (1,0), (0,-1), (-1,0)]
        islands = 0

        # once we find a single 1, dfs spreads to all the land connected to it
        # and adds the whole island to visit, so none of its cells get counted again
        def dfs(r,c):
            # stop: off the grid, already visited, or water
            if (r<0 or r==nRows or
                c<0 or c==nCols or
                (r,c) in visit or
                grid[r][c] == "0"):
                return

            visit.add((r,c))                # mark visited; grid stays unchanged
            # ALTERNATIVE (no visit set): sink the cell instead, a visited cell
            # becomes water, so the grid[r][c] == "0" check above stops on it
            # grid[r][c] = "0"              # (!) changes the input grid
            for dr, dc in directions:       # spread to the whole island
                dfs(r+dr, c+dc)
            # (!) no visit.remove(): reachability, not path counting
```

```
        for r in range(nRows):
            for c in range(nCols):
                # (!) check visit too, or every cell of an island starts a new count
                # (with the grid alternative, grid[r][c]=="1" alone is enough)
                if grid[r][c]=="1" and (r,c) not in visit:
                    dfs(r,c)
                    islands+=1

        return islands

# Time: O(rows * cols)
# Space: O(rows * cols) for visit + recursion (grid alternative: recursion only)
```

```python
### count-number-of-islands-bfs.py

"""
Count Number of Islands (NeetCode) -- another approach (BFS)
Problem statement: see count-number-of-islands-dfs.py
"""

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        nRows, nCols = len(grid), len(grid[0])
        directions = [(0,1), (1,0), (0,-1), (-1,0)]
        islands = 0
        visit = set()                   # cells already part of a counted island

        # once we find a single 1, bfs spreads level by level to all the land
        # connected to it and adds the whole island to visit
        def bfs(r,c):
            q = deque()
            q.append((r,c))
            visit.add((r,c))            # (!) mark the start too, not just neighbours

            while q:
                row, col = q.popleft()  # popleft = BFS (pop() would make it DFS)
                for dr, dc in directions:
                    newRow, newCol = row+dr, col+dc
                    # skip: off the grid, water, or already visited
                    if (newRow<0 or newCol<0 or
                        newRow==nRows or newCol==nCols or
                        grid[newRow][newCol] == "0" or
                        (newRow, newCol) in visit):
                        continue
                    q.append((newRow, newCol))
                    visit.add((newRow, newCol))    # (!) mark on push, so it's queued only once

        for r in range(nRows):
            for c in range(nCols):
                # (!) check visit too, or every cell of an island starts a new count
                if grid[r][c]=="1" and (r,c) not in visit:
                    bfs(r,c)
                    islands += 1

        return islands

# Time: O(rows * cols), Space: O(rows * cols) for visit + queue
```

```
### max-area-of-island-dfs.py

"""
Max Area of Island (NeetCode)
=============================

You are given a matrix grid where grid[i] is either a 0 (representing water)
or 1 (representing land).

An island is defined as a group of 1's connected horizontally or vertically.
You may assume all four edges of the grid are surrounded by water.

The area of an island is defined as the number of cells within the island.

Return the maximum area of an island in grid. If no island exists, return 0.

Example 1:
    Input: grid = [
        [0,1,1,0,1],
        [1,0,1,0,1],
        [0,1,1,0,1],
        [0,1,0,0,1]
      ]
    Output: 6
    Explanation: 1's cannot be connected diagonally, so the maximum area of
    the island is 6.

Constraints:
    1 <= grid.length, grid[i].length <= 50

Explanation:

    Same search as Count Number of Islands, two differences:
      - the search RETURNS a size: 1 for this cell + the sizes found in all
        4 directions (DFS), or +1 for every cell pushed (BFS)
      - keep the max size instead of adding 1 per island

    (!) grid holds ints (0/1) here, not strings ("0"/"1") like Number of Islands.
"""

class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        nRows, nCols = len(grid), len(grid[0])
        visit = set()                # cells already part of a measured island
        directions = [(0,1),(1,0),(0,-1),(-1,0)]

        # returns the size of the island containing (r,c), and marks all of it,
        # so no cell is counted twice
        def dfs(r,c):
            if r<0 or r==nRows or c<0 or c==nCols or grid[r][c]==0 or (r,c)in visit:
                return 0

            visit.add((r,c))

            area = 1                 # this cell
            # (!) add up ALL 4 directions; a return inside the loop only goes right
            for dr, dc in directions:
                area += dfs(r+dr, c+dc)
            return area

        area = 0
        for r in range(nRows):
            for c in range(nCols):
                # water and visited cells return 0, so calling dfs everywhere is safe
                area = max(area, dfs(r,c))

        return area

# Time: O(rows * cols), Space: O(rows * cols) for visit + recursion
```

```python
### max-area-of-island-bfs.py

"""
Max Area of Island (NeetCode) -- another approach (BFS)
Problem statement: see max-area-of-island-dfs.py
"""

class Solution:
    def maxAreaOfIsland(self, grid: List[List[int]]) -> int:
        nRows, nCols = len(grid), len(grid[0])
        visit = set()                # cells already part of a measured island
        directions = [(0,1),(1,0),(0,-1),(-1,0)]
        area = 0

        # once we find a single 1, bfs spreads to all the land connected to it,
        # adds the whole island to visit, and returns its size
        def bfs(r,c):
            q = deque()
            q.append((r,c))
            visit.add((r,c))         # mark the start too, not just neighbours
            res = 1                  # the start cell counts as 1

            while q:
                row, col = q.popleft()       # popleft = BFS (pop() would make it DFS)
                for dr, dc in directions:
                    # (!) neighbours of the POPPED cell (row, col), not the start (r, c)
                    newRow, newCol = row+dr, col+dc
                    # skip: off the grid, water (ints here, not "0"), or already visited
                    if (newRow<0 or newRow==nRows or
                        newCol<0 or newCol==nCols or
                        grid[newRow][newCol]==0 or
                        (newRow, newCol) in visit
                        ):
                        continue

                    q.append((newRow,newCol))
                    visit.add((newRow,newCol))  # (!) mark on push, so it's queued only once

                    res += 1         # each newly reached land cell adds 1 to the area
            return res

        maxArea = 0
        for r in range(nRows):
            for c in range(nCols):
                # (!) add 'and (r,c) not in visit': without it, bfs re-runs from every
                #    cell of an already-measured island (returns 1, so max is still right,
                #    but it's wasted work)
                if grid[r][c]==1 and (r,c) not in visit:
                    area = bfs(r,c)
                    maxArea = max(maxArea, area)

        return maxArea

# Time: O(rows * cols), Space: O(rows * cols) for visit + queue
```

```
### flood-fill.py

"""
Flood Fill (NeetCode)
=====================

You are given an image represented by an m x n grid of integers image, where
image[i][j] is the pixel value of the image. You are also given three integers
sr, sc, and color.

Perform a flood fill starting from the pixel image[sr][sc]: change the color
of the starting pixel, and of every pixel connected to it horizontally or
vertically that has the same original color, to color.

Return the modified image.

Example 1:
    Input:  image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2
    Output: [[2,2,2],[2,2,0],[2,0,1]]
    Explanation: Every 1 connected to the center pixel (1,1) becomes 2. The
    bottom-right 1 isn't connected (only diagonally), so it stays 1.

Example 2:
    Input:  image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0
    Output: [[0,0,0],[0,0,0]]
    Explanation: The starting pixel already has color 0, so nothing changes.

Constraints:
    m == image.length
    n == image[i].length
    1 <= m, n <= 50
    0 <= image[i][j], color < 2^16
    0 <= sr < m
    0 <= sc < n

Explanation:

    Same spreading search as Count Number of Islands, but from ONE given start
    instead of looping over every cell: "land" = originalColor, and marking a
    cell = recoloring it.

    Example 2 is the trap: when color == originalColor, a recolored cell still
    looks unvisited, so you need visit (or return early if the colors match).
"""

class Solution:
    def floodFill(self, image: List[List[int]], sr: int, sc: int, color: int) -> List[List[int]]:
        nRows, nCols = len(image), len(image[0])
        directions = [(0,1), (1,0), (0,-1), (-1,0)]
        originalColor = image[sr][sc]   # only cells with this color get filled
        visit = set()

        # spreads from (r,c) to every connected cell of originalColor and recolors it
        def dfs(r,c):
            # stop: off the grid, a different color, or already filled
            if r<0 or r==nRows or c<0 or c==nCols or image[r][c]!=originalColor or (r,c) in visit:
                return

            image[r][c] = color
            # (!) visit matters when color == originalColor: a recolored cell still
            #     matches originalColor, so without it the search never stops
            #     e.g. image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0
            visit.add((r,c))

            for dr,dc in directions:
                newRow, newCol = r+dr, c+dc
                dfs(newRow, newCol)

        dfs(sr, sc)                     # (!) start at the given pixel, not (0,0)
        return image

# Time: O(rows * cols), Space: O(rows * cols) for visit + recursion
```

```
### matrix-bfs.py

"""
Matrix Breadth-First Search: Shortest Path (NeetCode)
=====================================================

You are given a binary matrix grid where 0s represent land and 1s represent
rocks that can not be traversed.

Return the length of the shortest path from the top-left corner of grid to the
bottom-right corner such that all traversed cells are land cells. You may only
move vertically or horizontally through land cells.

Note:
    - If there is no such path, return -1.
    - The length of a path is the number of MOVES from the starting cell to the
      ending cell.

Example 1:
    Input: grid = [
        [0, 0, 0, 0],
        [1, 1, 0, 0],
        [0, 0, 0, 1],
        [0, 1, 0, 0]
      ]
    Output: 6

Explanation:

    Only BFS gives the shortest path here: it reaches cells in order of
    distance, so the first time the target is popped is the shortest way there.
    Process the queue one LEVEL at a time (snapshot len(queue)); every cell in
    a level is the same number of moves from the start.

      level 0: (0,0)                        length 0
      level 1: (0,1)                        length 1
      level 2: (0,2)                        length 2
      ...      each level is one move further, target popped at length 6

    Compare: Shortest Path in Binary Matrix counts CELLS and allows diagonals.
"""

class Solution:
    def shortestPath(self, grid: List[List[int]]) -> int:
        nRows, nCols = len(grid), len(grid[0])
        # (!) start on a rock -> no path at all
        if grid[0][0] == 1:
            return -1

        # BFS goes level by level: every cell in the queue right now is 'length'
        # moves from the start, so the first time we pop the target, that's the
        # shortest path
        def bfs(r,c):
            queue = deque()
            visit = set()
            directions = [(0,1),(1,0),(0,-1),(-1,0)]
            length = 0                      # moves taken to reach the current level
            queue.append((r,c))
            visit.add((r,c))

            while len(queue)>0:
                # (!) snapshot the level size; cells appended below belong to the next level
                for _ in range(len(queue)):
                    row, col = queue.popleft()
                    if (row,col) == (nRows-1, nCols-1):
                        return length       # covers a 1x1 grid too: 0 moves

                    for dr, dc in directions:
                        newRow, newCol = row+dr, col+dc
                        # skip: off the grid, rock, or already queued
                        if (newRow<0 or newRow==nRows or
                            newCol<0 or newCol==nCols or
                            grid[newRow][newCol]==1 or
                            (newRow, newCol) in visit):
                            continue

                        queue.append((newRow, newCol))
```

```
                        visit.add((newRow, newCol))    # mark on push, so it's queued once

                # once the for loop is finished meaning one level is finished.
                length += 1                 # (!) once per LEVEL (after the for), not per cell

            return -1                   # (!) queue emptied, target unreachable

        return bfs(0,0)

# Time: O(rows * cols), Space: O(rows * cols)
```

```
### shortest-path-in-binary-matrix.py

"""
Shortest Path in Binary Matrix (NeetCode)
=========================================

You are given an n x n binary matrix grid, return the length of the shortest
clear path in the matrix. If there is no clear path, return -1.

A clear path in a binary matrix is a path from the top-left cell (i.e., (0, 0))
to the bottom-right cell (i.e., (n - 1, n - 1)) such that:
    - All the visited cells of the path are 0.
    - All the adjacent cells of the path are 8-directionally connected (i.e.,
      they are different and they share an edge or a corner).

The length of a clear path is the number of visited cells of this path.

Example 1:
    Input: grid = [
        [0,1,0],
        [1,0,0],
        [1,1,0]
      ]
    Output: 3
    Explanation: (0,0) -> (1,1) -> (2,2), diagonal moves, 3 cells.

Example 2:
    Input: grid = [
        [1,0],
        [1,1]
      ]
    Output: -1

Constraints:
    n == grid.length
    n == grid[i].length
    1 <= n <= 100
    grid[i][j] is 0 or 1

Explanation:

    Same level-by-level BFS as the Matrix BFS lesson, two differences:
      - 8 directions (diagonals count as one step)
      - length counts CELLS, not moves, so it starts at 1 (a 1x1 grid -> 1)
"""

class Solution:
    def shortestPathBinaryMatrix(self, grid: List[List[int]]) -> int:
        nRows, nCols = len(grid), len(grid[0])
        # (!) start on a blocked cell -> no path
        if grid[0][0] == 1:
            return -1

        # BFS level by level: the first time we pop the bottom-right cell, that's the
        # shortest path. 8 directions here (diagonals allowed), unlike the matrix BFS lesson
        def bfs(r,c):
            directions = [(1,0),(1,1),(0,1),(1,-1),(0,-1),(-1,-1),(-1,0),(-1,1)]
            queue = deque()
            visit = set()

            queue.append((r,c))
            visit.add((r,c))

            length = 1                  # (!) counts CELLS on the path, so the start is 1

            while len(queue)>0:
                for _ in range(len(queue)):     # one full level per pass
                    row, col = queue.popleft()
                    if (row,col) == (nRows-1, nCols-1):
                        return length

                    for dr, dc in directions:
                        newRow, newCol = row+dr, col+dc
                        # skip: off the grid, blocked, or already queued
                        # (!) check the NEIGHBOUR in visit, not the popped cell
                        if (newRow < 0 or newRow==nRows or
                            newCol < 0 or newCol==nCols or
```

```
                            grid[newRow][newCol]==1 or
                            (newRow, newCol) in visit):
                            continue

                        queue.append((newRow, newCol))
                        visit.add((newRow, newCol))    # mark on push, so it's queued once

                # once for is exited that means one level is traversed.
                length += 1             # (!) once per LEVEL, after the for, not per cell

            return -1                   # target unreachable (or blocked)

        return bfs(0,0)

# Time: O(n^2), Space: O(n^2) for an n x n grid
```

```
### rotting-fruit.py

"""
Rotting Fruit (NeetCode)
========================

You are given a 2-D matrix grid. Each cell can have one of three possible
values:
    - 0 representing an empty cell
    - 1 representing a fresh fruit
    - 2 representing a rotten fruit

Every minute, if a fresh fruit is horizontally or vertically adjacent to a
rotten fruit, then the fresh fruit also becomes rotten.

Return the minimum number of minutes that must elapse until there are zero
fresh fruits remaining. If this state is impossible within the grid, return -1.

Example 1:
    Input: grid = [[1,1,0],[0,1,1],[0,1,2]]
    Output: 4

Example 2:
    Input: grid = [[1,0,1],[0,2,0],[1,0,1]]
    Output: -1
    Explanation: The fresh fruits in the corners are walled off by empty cells.

Constraints:
    1 <= grid.length, grid[i].length <= 10

Explanation:

    Multi-source BFS: queue up EVERY rotten fruit first, so the rot spreads from
    all of them at the same time. One BFS level = one minute.

      minute 0   minute 1   minute 2   (two rotten corners spreading together)
      2 1 1      2 2 1      2 2 2
      1 1 1  ->  2 1 2  ->  2 2 2      answer 2, not 4
      1 1 2      1 2 2      2 2 2

    `fresh` counts what's left: stop when it hits 0 (no extra minute for the
    last level), and if fruit is still fresh when the queue empties, it's -1.
"""

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        nRows, nCols = len(grid), len(grid[0])
        directions = [(0,1),(1,0),(0,-1),(-1,0)]

        queue = deque()
        visit = set()
        time = 0
        fresh =0

        # count fresh oranges, and queue up ALL rotten ones at once, so the rot
        # spreads from every one of them at the same time (multi-source BFS)
        for r in range(nRows):
            for c in range(nCols):
                if grid[r][c] == 1:
                    fresh += 1
                elif grid[r][c] == 2:
                    # find all rotten for which bfs will be run
                    queue.append((r,c))
                    visit.add((r,c))

        # one BFS level = one minute: everything in the queue now rots its neighbours
        # (!) `and fresh>0`: stop once nothing is left to rot, or the last level
        #    (oranges with no fresh neighbours) adds an extra minute
        while len(queue)>0 and fresh>0:
            for i in range(len(queue)):          # (!) snapshot: only this minute's oranges
                r, c = queue.popleft()
                for dr, dc in directions:
                    newRow, newCol = r+dr, c+dc
                    # (!) skip anything that ISN'T fresh (!= 1): empty (0) and rotten (2),
                    #    plus off the grid / already rotted
                    if (newRow<0 or newRow == nRows or
                        newCol<0 or newCol == nCols or
```

```
                        (newRow,newCol) in visit or
                        grid[newRow][newCol] != 1):
                        continue

                    queue.append((newRow, newCol))
                    visit.add((newRow, newCol))    # mark on push, so it rots (and counts) once
                    fresh -= 1                     # one fewer fresh orange
            time += 1                              # this minute's spread is done

            # ALTERNATIVE to 'and fresh>0' above: return as soon as the last one rots
            # if fresh == 0:
            #     return time

        # fresh left over = walled off by empty cells, the rot can never reach them
        if fresh==0:
            return time
        else:
            return -1

# Time: O(rows * cols), Space: O(rows * cols)
```

```python
### buy-and-sell-crypto.py

"""
Best Time to Buy and Sell Stock (NeetCode)
==========================================

You are given an integer array prices where prices[i] is the price of NeetCoin
on the ith day.

You may choose a single day to buy one NeetCoin and choose a different day in
the future to sell it.

Return the maximum profit you can achieve. You may choose to not make any
transactions, in which case the profit would be 0.

Example 1:
    Input:  prices = [10,1,5,6,7,1]
    Output: 6
    Explanation: Buy prices[1] and sell prices[4], profit = 7 - 1 = 6.

Example 2:
    Input:  prices = [10,8,7,5,2]
    Output: 0
    Explanation: No profitable transactions can be made, thus the max profit
    is 0.

Constraints:
    1 <= prices.length <= 100
    0 <= prices[i] <= 100
"""

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        l = 0            # buy day (lowest price seen so far)
        r = 1            # sell day (scans forward)
        max_profit = 0   # 0 if no profitable trade exists

        while r < len(prices):
            if prices[l] < prices[r]:
                # profitable: check if this sell beats the best so far
                profit = prices[r] - prices[l]
                max_profit = max(profit, max_profit)
            else:
                # found a cheaper (or equal) buy price -> move buy day here
                # safe to jump: any future sell is better off buying at r than at l
                l = r
            r += 1       # always advance sell day

        return max_profit
```

```
### longest-substring-without-duplicates.py

"""
Longest Substring Without Repeating Characters (NeetCode)
============================================================

Given a string s, find the length of the longest substring without duplicate
characters.

A substring is a contiguous sequence of characters within a string.

Example 1:
    Input:  s = "zxyzxyz"
    Output: 3
    Explanation: The string "xyz" is the longest without duplicate characters.

Example 2:
    Input:  s = "xxxx"
    Output: 1

Constraints:
    0 <= s.length <= 1000
    s may consist of printable ASCII characters.
"""

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        # Sliding window: keep s[l..r] free of repeats. O(N).
        charSet = set()         # chars in the current window

        l = 0
        r = 0                   # redundant: the for loop sets r
        res = 0

        for r in range(len(s)):             # r drives the loop; l only moves in the while
            while s[r] in charSet:          # check the NEW char (r)
                charSet.remove(s[l])        # shrink from the left until the dup is gone
                l += 1
            charSet.add(s[r])
            res = max(res, r - l + 1)       # window length; CountDistinctSlices(similar codility qu
estion) uses += here res += r-l+1

        return res
```

```
### cyclic-rotation.py


# keyboard_arrow_left
# An array A consisting of N integers is given. Rotation of the array means that each element is sh
ifted right by one index, and the last element of the array is moved to the first place. For exampl
e, the rotation of array A = [3, 8, 9, 7, 6] is [6, 3, 8, 9, 7] (elements are shifted right by one
index and 6 is moved to the first place).

# The goal is to rotate array A K times; that is, each element of A will be shifted to the right K
times.

# Write a function:

# def solution(A, K)

# that, given an array A consisting of N integers and an integer K, returns the array A rotated K t
imes.

# For example, given

#     A = [3, 8, 9, 7, 6]
#     K = 3
# the function should return [9, 7, 6, 3, 8]. Three rotations were made:

#     [3, 8, 9, 7, 6] -> [6, 3, 8, 9, 7]
#     [6, 3, 8, 9, 7] -> [7, 6, 3, 8, 9]
#     [7, 6, 3, 8, 9] -> [9, 7, 6, 3, 8]
# For another example, given

#     A = [0, 0, 0]
#     K = 1
# the function should return [0, 0, 0]

# Given

#     A = [1, 2, 3, 4]
#     K = 4
# the function should return [1, 2, 3, 4]

# Assume that:

# N and K are integers within the range [0..100];
# each element of array A is an integer within the range [-1,000..1,000].
# In your solution, focus on correctness. The performance of your solution will not be the focus of
 the assessment.

# Copyright 2009-2026 by Codility Limited. All Rights Reserved. Unauthorized copying, publication o
r disclosure prohibited.

def solution(arr, K):
    if not arr:
        return arr

    n = len(arr)
    K = K%n

    return arr[-K:]+arr[:-K]
```

```
### max-counters.py

"""
MaxCounters (Codility)

You are given N counters, initially set to 0, and you have two possible
operations on them:

    increase(X) - counter X is increased by 1,
    max counter - all counters are set to the maximum value of any counter.

A non-empty array A of M integers is given. This array represents consecutive
operations:

    if A[K] = X, such that 1 <= X <= N, then operation K is increase(X),
    if A[K] = N + 1 then operation K is max counter.

For example, given integer N = 5 and array A such that:

    A[0] = 3
    A[1] = 4
    A[2] = 4
    A[3] = 6
    A[4] = 1
    A[5] = 4
    A[6] = 4
the values of the counters after each consecutive operation will be:

    (0, 0, 1, 0, 0)
    (0, 0, 1, 1, 0)
    (0, 0, 1, 2, 0)
    (2, 2, 2, 2, 2)
    (3, 2, 2, 2, 2)
    (3, 2, 2, 3, 2)
    (3, 2, 2, 4, 2)
The goal is to calculate the value of every counter after all operations.

Write a function:

def solution(N, A)

that, given an integer N and a non-empty array A consisting of M integers, returns a sequence of in
tegers representing the values of the counters.

Result array should be returned as an array of integers.

For example, given:

    A[0] = 3
    A[1] = 4
    A[2] = 4
    A[3] = 6
    A[4] = 1
    A[5] = 4
    A[6] = 4
the function should return [3, 2, 2, 4, 2], as explained above.

Write an efficient algorithm for the following assumptions:

N and M are integers within the range [1..100,000];
each element of array A is an integer within the range [1..N + 1].
"""

# ****PERFORMANCE FAILS (77%)****
def solution(counter, array):
    # 1-indexed counters (ops use values 1..N), slot 0 unused
    counter_array = [0] * (counter + 1)
    max_counter_now = 0  # running max across all counters

    for counter_array_index in array:
        if counter_array_index == counter + 1:
            # "max counter" op: set ALL counters to current max
            # (!) O(N) rebuild every time -> O(N-M) worst case -> TLE
            counter_array = [max_counter_now] * (counter + 1)
        else:
            # "increase" op: bump that one counter
            counter_array[counter_array_index] += 1
```

```
            # track max so the next "max counter" op knows what to set
            max_counter_now = max(max_counter_now, counter_array[counter_array_index])

    return counter_array[1:]  # drop unused slot 0


def solution(N, A):
    MAX_COUNTER_OP = N + 1

    counters = [0] * (N + 1)    # 1-indexed; counters[0] unused
    current_max = 0             # highest value any counter has reached
    pending_min = 0             # every counter is at least this (set by last max-counter op, applie
d lazily)

    for op in A:
        if op == MAX_COUNTER_OP:
            # Don't rebuild the array (O(N)); just record the new minimum (O(1))
            pending_min = current_max
            continue

        # op is the counter to increment:
        # first catch it up to pending_min (if it missed a max op), then +1
        counters[op] = max(counters[op], pending_min) + 1
        current_max = max(current_max, counters[op])

    # Final cleanup: counters not touched since the last max op still hold stale values
    # e.g. stored [3, 0, 1, 4, 0] with pending_min=2 -> [3, 2, 2, 4, 2]
    result = []
    for value in counters[1:]:   # skip unused slot 0
        result.append(max(value, pending_min))
    return result
```

```
### arr-list-len-iterative.py

# ------------------------------------------------------------------------
# PROBLEM: Length of a linked list stored in an array  (Codility)
#
# A non-empty array A of N integers represents a linked list:
#   - the head node is at index 0
#   - the value of the node at index K is A[K]
#   - if the value is -1, that node is the last node (tail)
#   - otherwise the successor of node K is at index A[K]
#     (A[K] is always a valid index: 0 <= A[K] < N)
#
# Task: return the length of the list (number of nodes).
#
# EXAMPLE:
#   A[0] =  1
#   A[1] =  4
#   A[2] = -1
#   A[3] =  3
#   A[4] =  2
#
#              +--------------------+
#         +--+ |           +------+ |
#         |  v |           |      v |
#   A: [ 1 | 4 | -1 | 3 | 2 ]
#        0   1    2    3   4
#                 ^         |
#                 +---------+
#
#   The following list is constructed:
#   index: 0  ->  1  ->  4  ->  2
#   value: 1      4      2     -1 (tail)
#
#   Node at index 3 is never reached (not part of the list).
#   Answer = 4
#
# EDGE CASE: A = [-1]  -> head is also tail -> answer = 1
# ------------------------------------------------------------------------

# you can write to stdout for debugging purposes, e.g.
# print("this is a debug message")

def solution(A):
    # Guard: spec says A is non-empty, but handle it defensively
    if len(A)==0:
        return 0

    index = 0   # start at the head node (index 0)
    lenLl = 1   # head always exists, so count it

    # Follow successor pointers until we reach the tail (value -1)
    while A[index]!=-1:
        lenLl += 1          # moving to the next node adds one to the length
        index = A[index]    # successor of node at `index` is at A[index]

    # Time: O(N), each node visited once; Space: O(1)
    return lenLl
```

```
### arr-list-len-linked-list.py

"""
ArrListLen (Codility)
=====================

A non-empty array A consisting of N integers is given. Array A represents a
linked list. A list is constructed from this array as follows:

    - the first node (the head) is located at index 0;
    - the value of a node located at index K is A[K];
    - if the value of a node is -1 then it is the last node of the list;
    - otherwise, the successor of a node located at index K is located at
      index A[K] (you can assume that A[K] is a valid index, that is
      0 <= A[K] < N).

For example, for array A such that:

    A[0] =  1
    A[1] =  4
    A[2] = -1
    A[3] =  3
    A[4] =  2

the following list is constructed:

    - the first node (the head) is located at index 0 and has a value of 1;
    - the second node is located at index 1 and has a value of 4;
    - the third node is located at index 4 and has a value of 2;
    - the fourth node is located at index 2 and has a value of -1.

Write a function:

def solution(A)

that, given a non-empty array A consisting of N integers, returns the length
of the list constructed from A in the above manner.

For example, given array A such that A = [1, 4, -1, 3, 2] the function should
return 4, as explained in the example above.

Assume that:

N is an integer within the range [1..200,000];
each element of array A is an integer within the range [-1..N-1];
it will always be possible to construct the list and its length will be
finite.
"""

# *** HITS TIMEOUT coz of push so use doublyLL***
class Node:
    def __init__(self,val):
        self.val = val
        self.next = None          # singly linked: only a 'next' pointer

class LinkedList:
    def __init__(self):
        self.head = Node(0)       # dummy/sentinel node, not counted in size
        self.size = 0

    def push(self, val):
        # NOTE: walks head -> end on every push = O(N) per push, O(N^2) total.
        #   Correct, but may time out on large Codility tests.
        curr = self.head
        for i in range(self.size):
            curr = curr.next
        curr.next = Node(val)
        self.size += 1

    def print_ll(self):
        curr = self.head.next
        for i in range(self.size):
            print(curr.val, end = "=>")
            curr = curr.next
        print()

    def size_of_ll(self):
```

```
        return self.size

def solution(A):
    if len(A)==0:
        return 0

    l = LinkedList()
    index = 0                   # start at the head (index 0)
    l.push(A[index])            # head node is always part of the list

    # FIX: follow the pointers until the current node's value is -1.
    while A[index] != -1:
        index = A[index]        # jump to the successor
        l.push(A[index])        # add that node's value

    # l.print_ll()                # debug only; stdout is ignored in scoring
    return l.size_of_ll()       # Time: O(N^2) due to push, Space: O(N)
```

```
### perm-missing-elem.py

"""
An array A consisting of N different integers is given. The array contains integers in the range [1
..(N + 1)], which means that exactly one element is missing.
Your goal is to find that missing element.
Write a function:
def solution(A)
that, given an array A, returns the value of the missing element.
For example, given array A such that:
  A[0] = 2
  A[1] = 3
  A[2] = 1
  A[3] = 5
the function should return 4, as it is the missing element.
Write an efficient algorithm for the following assumptions:

N is an integer within the range [0..100,000];
the elements of A are all distinct;
each element of array A is an integer within the range [1..(N + 1)].
"""

def solution(arr):
    n = len(arr)
    element_dict = {}

    # n+2, not n+1: range is 1..N+1 inclusive (N+1 values)
    for i in range(1, n+2):
        element_dict[i] = False

    # mark seen
    for element in arr:
        element_dict[element] = True

    # the one still False is missing; N=0 -> {1: False} -> returns 1
    for key, value in element_dict.items():
        if value == False:          # 'not value' is more idiomatic
            return key
```

```
### frog-river-one.py

"""
A small frog wants to get to the other side of a river. The frog is initially located on one bank o
f the river (position 0) and wants to get to the opposite bank (position X+1). Leaves fall from a t
ree onto the surface of the river.

You are given an array A consisting of N integers representing the falling leaves. A[K] represents
the position where one leaf falls at time K, measured in seconds.

The goal is to find the earliest time when the frog can jump to the other side of the river. The fr
og can cross only when leaves appear at every position across the river from 1 to X (that is, we wa
nt to find the earliest moment when all the positions from 1 to X are covered by leaves). You may a
ssume that the speed of the current in the river is negligibly small, i.e. the leaves do not change
 their positions once they fall in the river.

For example, you are given integer X = 5 and array A such that:

  A[0] = 1
  A[1] = 3
  A[2] = 1
  A[3] = 4
  A[4] = 2
  A[5] = 3
  A[6] = 5
  A[7] = 4
In second 6, a leaf falls into position 5. This is the earliest time when leaves appear in every po
sition across the river.

Write a function:

def solution(X, A)

that, given a non-empty array A consisting of N integers and integer X, returns the earliest time w
hen the frog can jump to the other side of the river.

If the frog is never able to jump to the other side of the river, the function should return -1.

For example, given X = 5 and array A such that:

  A[0] = 1
  A[1] = 3
  A[2] = 1
  A[3] = 4
  A[4] = 2
  A[5] = 3
  A[6] = 5
  A[7] = 4
the function should return 6, as explained above.

Write an efficient algorithm for the following assumptions:

N and X are integers within the range [1..100,000];
each element of array A is an integer within the range [1..X].
"""


def solution(posn_to_reach, arr):
    # positions are 1..X (1-indexed), so size X+1 lets us use covered[posn] directly
    # index 0 is unused (that's the riverbank, no leaf ever lands there)
    covered = [False] * (posn_to_reach + 1)

    # count down instead of re-scanning 'covered' each second -> keeps it O(N)
    posns_with_no_leaves = posn_to_reach

    for sec, posn in enumerate(arr):    # sec = index = time, posn = where leaf lands
        if not covered[posn]:           # duplicates are ignored, they don't reduce the count
            covered[posn] = True
            posns_with_no_leaves -= 1
            if posns_with_no_leaves == 0:  # last gap just filled -> earliest crossing time
                return sec

    return -1  # some position never got a leaf
```

```
### missing-integer.py

"""
Task description

keyboard_arrow_left
This is a demo task.

Write a function:

def solution(A)

that, given an array A of N integers, returns the smallest positive integer (greater than 0) that d
oes not occur in A.

For example, given A = [1, 3, 6, 4, 1, 2], the function should return 5.

Given A = [1, 2, 3], the function should return 4.

Given A = [−1, −3], the function should return 1.

Write an efficient algorithm for the following assumptions:

N is an integer within the range [1..100,000];
each element of array A is an integer within the range [−1,000,000..1,000,000].
"""

def solution(array):
    numsSet = set()
    for num in array:
        numsSet.add(num)

    for num in range(1,1000000):
        if num not in numsSet:
            return num


def solution2(A):
    # MissingInteger / LC 41 First Missing Positive. O(N) time, O(N) space.
    #
    # Why the answer is always in [1, N+1], where N = len(A):
    #   - N elements can cover at most N distinct positives, i.e. at most 1..N.
    #   - If all of 1..N are present -> answer is N+1.
    #       e.g. [1..1000], N=1000 -> 1001
    #   - Otherwise something in 1..N is missing -> answer <= N.
    #       e.g. [1, 3, 6, 4, 1, 2], N=6 -> 5
    #   - An answer of N+2 would need 1..N+1 present = N+1 elements. Impossible.
    #   So values < 1 or > N never affect the answer, and the input range
    #   [-1e6, 1e6] is a distraction; only N matters.
    #
    # Edge cases: all negatives -> 1, duplicates, full 1..N -> N+1.
    s = set(A)
    for i in range(1, len(A) + 2):   # checks 1..N+1 (range end is exclusive)
        if i not in s:
            return i
```

```
### binary-gap.py

"""
BinaryGap (Codility)

A binary gap within a positive integer N is any maximal sequence of consecutive
zeros that is surrounded by ones at both ends in the binary representation of
N.

For example, number 9 has binary representation 1001 and contains a binary gap
of length 2. The number 529 has binary representation 1000010001 and contains
two binary gaps: one of length 4 and one of length 3. The number 20 has binary
representation 10100 and contains one binary gap of length 1. The number 15 has
binary representation 1111 and has no binary gaps. The number 32 has binary
representation 100000 and has no binary gaps.

Write a function:

def solution(N)

that, given a positive integer N, returns the length of its longest binary gap.
The function should return 0 if N doesn't contain a binary gap.

For example, given N = 1041 the function should return 5, because N has binary
representation 10000010001 and so its longest binary gap is of length 5. Given
N = 32 the function should return 0, because N has binary representation
'100000' and thus no binary gaps.

Write an efficient algorithm for the following assumptions:

N is an integer within the range [1..2,147,483,647].

My test values (N, binary, expected):
    9    1001        2
    529  1000010001  4
    20   10100       1
    15   1111        0
"""

def solution(N):
    binary = bin(N)
    binary = binary[2:]
    # print(binary)

    max_count = 0

    for index, char in enumerate(binary):
        if char == "1":
            count = 0
            if index+1 < len(binary):
                curr = index+1
                while curr < len(binary):
                    if binary[curr] == "0":
                        count+=1
                    else:
                        # only update max_count when 1 is encountered
                        max_count = max(count, max_count)
                        break
                    curr += 1

    # print(f"n ={N}, binary = {binary}, max_count = {max_count}")
    return max_count
```

```
### count-bounded-slices.py

"""
CountBoundedSlices (Codility)
=============================

An integer K and a non-empty array A consisting of N integers are given.

A pair of integers (P, Q), such that 0 <= P <= Q < N, is called a slice of
array A.

A bounded slice is a slice in which the difference between the maximum and
minimum values in the slice is less than or equal to K. More precisely it is a
slice, such that max(A[P], A[P + 1], ..., A[Q]) - min(A[P], A[P + 1], ..., A[Q])
<= K.

The goal is to calculate the number of bounded slices.

For example, consider K = 2 and array A such that:

    A[0] = 3
    A[1] = 5
    A[2] = 7
    A[3] = 6
    A[4] = 3

There are exactly nine bounded slices: (0, 0), (0, 1), (1, 1), (1, 2), (1, 3),
(2, 2), (2, 3), (3, 3), (4, 4).

Write a function:

def solution(K, A)

that, given an integer K and a non-empty array A of N integers, returns the
number of bounded slices of array A.

If the number of bounded slices is greater than 1,000,000,000, the function
should return 1,000,000,000.

For example, given K = 2 and array A as above, the function should return 9,
as explained above.

Write an efficient algorithm for the following assumptions:

N is an integer within the range [1..100,000];
K is an integer within the range [0..1,000,000,000];
each element of array A is an integer within the range
[-1,000,000,000..1,000,000,000].
"""

# ***** PERFORMANCE FAILS ****

def solution(K, arr):
    arr_len = len(arr)
    count = 0
    # max_val = arr[0]
    # min_val = arr[0]

    for i in range(arr_len):
        max_val = arr[i]
        min_val = arr[i]

        for j in range(i, arr_len):
            max_val = max(max_val, arr[j])
            min_val = min(min_val, arr[j])

            diff = max_val-min_val
            if diff <= K:
                count += 1
            else:
              break

    return count
```

```
### count-distinct-slices.py

"""
Task description

keyboard_arrow_left
An integer M and a non-empty array A consisting of N non-negative integers are given. All integers
in array A are less than or equal to M.

A pair of integers (P, Q), such that 0 <= P <= Q < N, is called a slice of array A. The slice consi
sts of the elements A[P], A[P + 1], ..., A[Q]. A distinct slice is a slice consisting of only uniqu
e numbers. That is, no individual number occurs more than once in the slice.

For example, consider integer M = 6 and array A such that:

    A[0] = 3
    A[1] = 4
    A[2] = 5
    A[3] = 5
    A[4] = 2
There are exactly nine distinct slices: (0, 0), (0, 1), (0, 2), (1, 1), (1, 2), (2, 2), (3, 3), (3,
 4) and (4, 4).

The goal is to calculate the number of distinct slices.

Write a function:

def solution(M, A)

that, given an integer M and a non-empty array A consisting of N integers, returns the number of di
stinct slices.

If the number of distinct slices is greater than 1,000,000,000, the function should return 1,000,00
0,000.

For example, given integer M = 6 and array A such that:

    A[0] = 3
    A[1] = 4
    A[2] = 5
    A[3] = 5
    A[4] = 2
the function should return 9, as explained above.

Write an efficient algorithm for the following assumptions:

N is an integer within the range [1..100,000];
M is an integer within the range [0..100,000];
each element of array A is an integer within the range [0..M].
"""

def solution(max_val, arr):
    # Brute force: for each start l, extend r until a duplicate appears.
    # O(N^2) is correct but fails perf; use a sliding window for O(N).
    res = []

    for l in range(len(arr)):
        new_set = set()             # values in the current slice arr[l..r]
        new_set.add(arr[l])
        res.append((l,l))           # a single element is always a valid slice
        for r in range(l+1, len(arr)):
            if arr[r] not in new_set:
                new_set.add(arr[r])
                res.append((l,r))   # record EVERY valid extension, not only at a dup
            else:
                break               # dup found: every longer slice from l is invalid too

    # print(res)
    return len(res)                 # missing the 1e9 cap from the spec


def solution(M, array):
    # Sliding window: keep array[l..r] all-distinct. O(N).
    l = 0
    r = 0                   # redundant: the for loop sets r
    res = 0
    numSet = set()          # values in the current window
```

```
    for r in range(len(array)):          # r drives the loop; l only moves in the while
        while array[r] in numSet:        # check the NEW element (r), not array[l]
            numSet.remove(array[l])      # shrink from the left until the dup is gone
            l += 1
        numSet.add(array[r])

        # window [l..r] is distinct, so every slice ending at r is distinct:
        # starts l, l+1, ..., r  ->  r - l + 1 slices
        # (each slice is counted once, by its end index)
        res = res + r - l + 1

        if res > 1000000000:             # spec: cap at 1e9
            return 1000000000

    return res
```

```
### tape-equilibrium.py

"""
A non-empty array A consisting of N integers is given. Array A represents numbers on a tape.

Any integer P, such that 0 < P < N, splits this tape into two non-empty parts: A[0], A[1], ..., A[P
 - 1] and A[P], A[P + 1], ..., A[N - 1].

The difference between the two parts is the value of: |(A[0] + A[1] + ... + A[P - 1]) - (A[P] + A[P
 + 1] + ... + A[N - 1])|

In other words, it is the absolute difference between the sum of the first part and the sum of the
second part.

For example, consider array A such that:

  A[0] = 3
  A[1] = 1
  A[2] = 2
  A[3] = 4
  A[4] = 3
We can split this tape in four places:

P = 1, difference = |3 - 10| = 7
P = 2, difference = |4 - 9| = 5
P = 3, difference = |6 - 7| = 1
P = 4, difference = |10 - 3| = 7
Write a function:

class Solution { public int solution(int[] A); }

that, given a non-empty array A of N integers, returns the minimal difference that can be achieved.

For example, given:

  A[0] = 3
  A[1] = 1
  A[2] = 2
  A[3] = 4
  A[4] = 3
the function should return 1, as explained above.

Write an efficient algorithm for the following assumptions:

N is an integer within the range [2..100,000];
each element of array A is an integer within the range [-1,000..1,000].
"""

def solution(arr):
    total = sum(arr)  # once; sum() on slices in the loop = O(N^2), fails perf
    left_sum = 0
    min_diff = float('inf')  # not 0, else it always wins

    # N-1, not N: both parts must be non-empty.
    # range(N) allows an empty right part -> [-1000, 1000] returns 0, not 2000
    for i in range(len(arr) - 1):
        left_sum += arr[i]                      # update before diff
        right_sum = total - left_sum
        min_diff = min(min_diff, abs(right_sum - left_sum))  # abs: negatives allowed

    return min_diff
```

```
### genomic-range-query.py

"""
GenomicRangeQuery (Codility - Prefix Sums)

A DNA sequence can be represented as a string consisting of the letters A, C, G and T,
which correspond to the types of successive nucleotides in the sequence. Each nucleotide
has an impact factor, which is an integer. Nucleotides of types A, C, G and T have impact
factors of 1, 2, 3 and 4, respectively. You are going to answer several queries of the form:
What is the minimal impact factor of nucleotides contained in a particular part of the given
DNA sequence?

The DNA sequence is given as a non-empty string S = S[0]S[1]...S[N-1] consisting of N
characters. There are M queries, which are given in non-empty arrays P and Q, each consisting
of M integers. The K-th query (0 <= K < M) requires you to find the minimal impact factor of
nucleotides contained in the DNA sequence between positions P[K] and Q[K] (inclusive).

For example, consider string S = CAGCCTA and arrays P, Q such that:
    P[0] = 2    Q[0] = 4
    P[1] = 5    Q[1] = 5
    P[2] = 0    Q[2] = 6

The answers to these M = 3 queries are as follows:
* The part of the DNA between positions 2 and 4 contains nucleotides G and C (twice),
  whose impact factors are 3 and 2 respectively, so the answer is 2.
* The part between positions 5 and 5 contains a single nucleotide T, whose impact factor
  is 4, so the answer is 4.
* The part between positions 0 and 6 (the whole string) contains all nucleotides, in
  particular nucleotide A whose impact factor is 1, so the answer is 1.

Write a function:
    def solution(S, P, Q)
that, given a non-empty string S consisting of N characters and two non-empty arrays P and Q
consisting of M integers, returns an array consisting of M integers specifying the consecutive
answers to all queries.

For example, given S = CAGCCTA and P, Q as above, the function should return [2, 4, 1].

Write an efficient algorithm for the following assumptions:
* N is an integer within the range [1..100,000];
* M is an integer within the range [1..50,000];
* each element of arrays P and Q is an integer within the range [0..N - 1];
* P[K] <= Q[K], where 0 <= K < M;
* string S consists only of upper-case English letters A, C, G, T.
"""


# ------------------------------------------------------------------------
# Approach 1: Brute force (my first attempt) -- correct, but fails performance
# ------------------------------------------------------------------------
def solution_brute(input_string, P, Q):
    impact_dict = {"A": 1, "C": 2, "G": 3, "T": 4}

    res = []
    for i in range(len(P)):
        # (!) Slice end is Q[i]+1 because Q is inclusive and Python slices are exclusive.
        # (!) Slicing also copies the substring, which is an extra O(len) per query.
        dna_sequence = input_string[P[i]:Q[i] + 1]

        # Scan the whole range for the min impact factor
        min_nucleotide_val = float('inf')
        for dna_nucleotide in dna_sequence:
            min_nucleotide_val = min(min_nucleotide_val, impact_dict[dna_nucleotide])
            # (could break early once it hits 1, but the worst case is still the same)

        res.append(min_nucleotide_val)

    return res

# (!) Why it fails: each query scans up to N chars, so the total is O(N * M).
#     Worst case 100,000 * 50,000 = 5 billion ops. TLE.
# Time: O(N * M), Space: O(N) for the slice


# ------------------------------------------------------------------------
# Approach 2: Prefix counts per nucleotide -- O(N + M)
# ------------------------------------------------------------------------
```

```python
# Key idea: the answer only depends on WHICH nucleotides appear in the range,
# not where. With only 4 types, precompute running counts for each one. Then
# "is there an A in [p, q]?" is a single subtraction.
def solution(S, P, Q):
    n = len(S)

    # prefix[c][i] = count of nucleotide c in S[0:i] (the first i chars)
    # (!) Size n+1 with prefix[c][0] = 0, so the range count is a clean subtraction.
    prefix = {c: [0] * (n + 1) for c in "ACGT"}

    for i, ch in enumerate(S):
        # Carry forward all 4 counts, then bump the current char
        for c in "ACGT":
            prefix[c][i + 1] = prefix[c][i]
        prefix[ch][i + 1] += 1

    res = []
    for p, q in zip(P, Q):
        # Count of c in S[p..q] inclusive = prefix[c][q+1] - prefix[c][p]
        # (!) q+1, not q. Q is inclusive, prefix is exclusive.
        # Check in impact order: the first one present is the minimum.
        if prefix["A"][q + 1] - prefix["A"][p] > 0:
            res.append(1)
        elif prefix["C"][q + 1] - prefix["C"][p] > 0:
            res.append(2)
        elif prefix["G"][q + 1] - prefix["G"][p] > 0:
            res.append(3)
        else:
            res.append(4)  # (!) No A/C/G means it must be all T. No need to check.

    return res

# Trace for S = "CAGCCTA", query (2, 4) -> "GCC":
#   A: prefix[5] - prefix[2] = 1 - 1 = 0  -> no A
#   C: prefix[5] - prefix[2] = 3 - 1 = 2  -> answer 2
#
# Time: O(N + M)  (4*N to build + O(1) per query)
# Space: O(N)


if __name__ == "__main__":
    S, P, Q = "CAGCCTA", [2, 5, 0], [4, 5, 6]
    print(solution_brute(S, P, Q))  # [2, 4, 1]
    print(solution(S, P, Q))        # [2, 4, 1]
```

```
### passing-cars.py

"""
A non-empty array A consisting of N integers is given. The consecutive elements of array A represen
t consecutive cars on a road.

Array A contains only 0s and/or 1s:

0 represents a car traveling east,
1 represents a car traveling west.
The goal is to count passing cars. We say that a pair of cars (P, Q), where 0 <= P < Q < N, is pass
ing when P is traveling to the east and Q is traveling to the west.

For example, consider array A such that:

  A[0] = 0
  A[1] = 1
  A[2] = 0
  A[3] = 1
  A[4] = 1
We have five pairs of passing cars: (0, 1), (0, 3), (0, 4), (2, 3), (2, 4).

Write a function:

def solution(A)

that, given a non-empty array A of N integers, returns the number of pairs of passing cars.

The function should return -1 if the number of pairs of passing cars exceeds 1,000,000,000.

For example, given:

  A[0] = 0
  A[1] = 1
  A[2] = 0
  A[3] = 1
  A[4] = 1
the function should return 5, as explained above.

Write an efficient algorithm for the following assumptions:

N is an integer within the range [1..100,000];
each element of array A is an integer that can have one of the following values: 0, 1.
"""

def solution(arr):
    # BRUTE FORCE: O(N^2) time, O(pairs) space. Correct, but TLEs on large N.
    #
    # My thinking:
    #   A passing pair is (P, Q) with P < Q, arr[P] == 0 (east), arr[Q] == 1 (west).
    #   So for every 0 at index l, scan everything to its right and
    #   record each 1 as a pair (l, r). The answer is the number of pairs.
    res = []

    for l in range(len(arr)):
        if arr[l] == 0:
            # Eastbound car: pair it with every westbound car to its right
            for r in range(l + 1, len(arr)):  # l+1 enforces P < Q
                if arr[r] == 1:
                    res.append((l, r))
        else:
            # Intent: "this is a 1, it can't start a pair, so skip it and move on".
            # (!) USELESS LINE. `for l in range(...)` rebinds l to the next value at
            #    the start of every iteration, so this increment is overwritten
            #    immediately. Changing the loop variable doesn't affect range().
            #    (In Go's `for l := 0; l < n; l++`, l++ inside WOULD skip an index.)
            # Why the code still works: skipping a 1 and moving to the next index
            # is exactly what the for loop does anyway. The else branch could be
            # deleted with no change in behavior.
            l += 1

    # (!) Storing every pair is wasteful. Only the count is needed, so `count += 1` would do.
    # (!) Missing requirement: return -1 if the count exceeds 1,000,000,000.
    return len(res)

# Why it TLEs: in the worst case (half 0s followed by half 1s) that's ~N^2/4 checks.
#    N = 100,000 -> ~2.5 billion operations.
```

```
#
# Optimal O(N) idea (see the other version): flip the view. Instead of each 0
# looking right for 1s, each 1 looks LEFT. It pairs with every 0 seen so far,
# so keep a running `east` count and do `pairs += east` whenever you hit a 1.

def solution(A):
    # Number of eastbound cars (0s) seen so far.
    # Every one of them is behind (to the left of) the current position,
    # so any westbound car we meet from here on will pass all of them.
    east = 0

    # Total number of passing pairs found so far.
    pairs = 0

    for car in A:
        if car == 0:
            # Eastbound car: it can't pass anything to its left (pairs need P < Q
            # with P going east), but it WILL pass every westbound car to its right.
            # Record it so future 1s can count it.
            east += 1
        else:
            # Westbound car: it forms a pair with every eastbound car before it.
            # That's exactly `east` new pairs, no need to look back at them individually.
            pairs += east

            # Problem requires -1 once the count exceeds 1e9.
            # Checking here lets us stop early instead of finishing the loop.
            if pairs > 1_000_000_000:
                return -1

    return pairs
```

```
### min-avg-two-slice.py

"""
A non-empty array A consisting of N integers is given. A pair of integers (P, Q), such that 0 <= P
< Q < N, is called a slice of array A (notice that the slice contains at least two elements). The a
verage of a slice (P, Q) is the sum of A[P] + A[P + 1] + ... + A[Q] divided by the length of the sl
ice. To be precise, the average equals (A[P] + A[P + 1] + ... + A[Q]) / (Q - P + 1).

For example, array A such that:

    A[0] = 4
    A[1] = 2
    A[2] = 2
    A[3] = 5
    A[4] = 1
    A[5] = 5
    A[6] = 8
contains the following example slices:

slice (1, 2), whose average is (2 + 2) / 2 = 2;
slice (3, 4), whose average is (5 + 1) / 2 = 3;
slice (1, 4), whose average is (2 + 2 + 5 + 1) / 4 = 2.5.
The goal is to find the starting position of a slice whose average is minimal.

Write a function:

def solution(A)

that, given a non-empty array A consisting of N integers, returns the starting position of the slic
e with the minimal average. If there is more than one slice with a minimal average, you should retu
rn the smallest starting position of such a slice.

For example, given array A such that:

    A[0] = 4
    A[1] = 2
    A[2] = 2
    A[3] = 5
    A[4] = 1
    A[5] = 5
    A[6] = 8
the function should return 1, as explained above.

Write an efficient algorithm for the following assumptions:

N is an integer within the range [2..100,000];
each element of array A is an integer within the range [-10,000..10,000].
"""

def solutionBrute(arr):
    min_sum_avg = float('inf')
    min_sum_start = 0

    for l in range(0, len(arr)):
        for r in range(l+1, len(arr)):
            new_avg = sum(arr[l:r+1])/(r-l+1)
            if new_avg < min_sum_avg:
                min_sum_start = l
                min_sum_avg = new_avg

    return min_sum_start

def solution(A):
    # KEY MATH FACT: the min-average slice always has length 2 or 3.
    # Why: a slice of length >= 4 can be split into smaller slices, each of length >= 2.
    #   The average of a whole is never less than the smallest average of its parts.
    #   (If you split into parts with averages a and b, the whole lies between a and b.)
    #   So a part's average is <= the whole's. Keep splitting and you end up at length 2 or 3.
    #   Length 3 can't be split into slices of length >= 2, so it has to be checked too.
    n = len(A)
    min_avg = (A[0] + A[1]) / 2
    min_start = 0

    for i in range(n - 1):
        # Length-2 slice starting at i
        avg2 = (A[i] + A[i + 1]) / 2
        if avg2 < min_avg:
```

```
            min_avg = avg2
            min_start = i

        # Length-3 slice starting at i
        # (!) Bounds check: a 3-slice needs i+2 < n
        if i + 2 < n:
            avg3 = (A[i] + A[i + 1] + A[i + 2]) / 3
            if avg3 < min_avg:
                min_avg = avg3
                min_start = i

    # (!) Strict < plus a left-to-right scan returns the smallest start on ties
    return min_start

# Time: O(N), Space: O(1)
```